

u-boot 1.1.2 Porting Guide to PXA27x

v 0.9

2006.2.8

**김기오 (gurugio@gmail_nospam.com)
<http://www.asmlove.co.kr>**

알림.

이 문서는 개인적인 참고 자료의 모음과 연구실 내부에서 신입생 교육에 사용될 개인적인 자료입니다. 혹시 저작권이나 기타 위반 사항 등이 발견되면 언제라도 연락 주시면 즉시 바로잡도록 하겠습니다. 이 문서는 개인 적인 참고 용도로만 사용될 수 있습니다.

이 문서는 **GNU Free Documentation License**를 따릅니다.

김기오 (gurugio@gmail_nospam.com.nospam)

update.

2005년 12월 18일 kldp.wiki 등록

2005년 12월 20일 레지스터 설명, 관련 코드 추가 v 0.01

2005년 2월 8일 관련 코드 설명, 오류 수정 v0.9

플래시 프로그래머 포팅

먼저 인텔에서 jflashmm 프로그램을 다운받는다. 이 프로그램은 윈도우 전용이다. 먼저 윈도우에 설치해서 보드를 인식하는지 테스트하고 소스를 리눅스로 옮기고 Makefile을 만들어서 포팅한다.

JFlashmm 다운로드 주소

<http://www.intel.com/design/pca/applicationsprocessors/swsup/JFlashMM.htm>

인텔은 다운로드 주소를 수시로 바꾸므로 인텔 홈페이지에서 검색한다.

JFlash_MM_V5_01_007.exe 이라는 프로그램을 설치하면 디렉토리에 Flash_88XX~~.dat 라는 설정 파일들과 Source.zip 소스 파일등이 있다. 프로그램을 설치한 후에 RelNode_JFlashmm.htm 을 읽으면서 giveio.inf 드라이버 파일을 윈도우에 설치한다. 그래야 프로그램이 정상적으로 작동된다. 그리고 보드를 만들 때 프로세서와 플래쉬, 램등 최소한의 장치만 조립하고 윈도우에 설치된 JFlashmm로 동작을 테스트한다. 그런데 한가지 빠진 파일이 있는데 bulbcx.dat 파일이다. <http://www.linuxgazette.com/node/9786> 문서는 2.4 커널을 mainstone 보드에 포팅하는 문서인데 bulbcx.dat 에 대해 다음과 같이 말하고 있다.

Connect the parallel port connector to MAIN JTAG (J4) and parallel port on your computer. Secondly copy the bulbcx.dat and blob-smc files to the folder where the jflash is installed.

Ex: C:\Program Files\Intel Corporation\JFlash_MM\

Bulbcx.dat: Has the information about the processor and the board. And it is provided by the board manufacturer.

이 파일을 인터넷에서 구해서 jflash 폴더에 복사한다. 그리고 다음과 같이 실행해본다.

```
JFlash Window Console

C:\JFlash_MM>JFlashmm.exe bulbcx testfile.bin i

JFLASH Version 5.01.007
COPYRIGHT (C) 2000 - 2003 Intel Corporation

PLATFORM SELECTION:
Processor=          PXA27x
Development System= Mainstone
Data Version=       1.00.002

PXA27x revision C0
Found flash type: 28F128K3

Program successful completion.
Processor and Flash Memory Identified.
No programming operation performed.

C:\JFlash_MM>JFlashmm.exe bulbcx testfile.bin v

JFLASH Version 5.01.007
COPYRIGHT (C) 2000 - 2003 Intel Corporation

PLATFORM SELECTION:
Processor=          PXA27x
Development System= Mainstone
Data Version=       1.00.002

PXA27x revision C0
Found flash type: 28F128K3

Starting Verify
verify error at address = 0 exp_dat = 11111111 act_dat = 111111

C:\JFlash_MM>
```

i 옵션으로 실행해서 플래시와 프로세서를 인식하는지를 확인한다. 여기까지 된다면 프로세서와 플래시 메모리는 동작하는 것이다.

다음으로 Source.zip과 *.dat 설정 파일들을 리눅스로 복사한다.

* jflashmm을 포팅하는 것은 수정할 양이 많으므로 일단 하이버스에서 제공한 시디에 있는 jflashmm을 사용한다. 추후 참고 자료를 구하거나 시간이 날 때 다시 분석한다. 하이버스에서 제공하는 설정파일 pxa27x32.dat는 bulbcx.dat와 같은 파일이다. 따라서 bulbcx.dat를 사용한다.

** 리눅스로 복사한 소스 중 Global_variable.h에서 41줄에 있는 CABLE_TYPES를 Parallel_Jtag에서Insight_Jtag으로 수정한다. 원래 인텔에서 제공하는 jflashmm 소스를 보면 이 부분이 Insight_Jtag으로 되어있어서 이렇게 고친다. Parallel Jtag 타입으로 컴파일하면 인식을 못하고 JTag에서 나오는 신호가 다르다.

```
gurutio@gioserver:/home/SKKU/bulverde/jflashmm
34 char flash_data_filename[MAX_IN_LENGTH] = "Flash_18_2_32.dat"; // Global variable for the flash data file
35 char int_data_filename[MAX_IN_LENGTH] = "DBPXA250_INTEGRITY.DAT"; // global integrity data file
36
37 char VERSION_LOCK[11] = "VL000000001";
38 char FLASH_VERSION_LOCK[11] = "VLF00000001";
39
40 /* fixed by HYBUS */
41 //CABLE_TYPES CableType = Parallel_Jtag; // Global variable for specifying the Cable type
42
43 /* In original jflashmm sources, CABLE_TYPES is Insight_Jtag
44 * fixed by Gi-Oh Kim */
45 CABLE_TYPES CableType = Insight_Jtag;
46
47
48 DWORD ChipSelect0 = 0; // Global variable for chip select 0
49 DWORD ChipSelect1 = 0; // Global variable for chip select 1
50 DWORD ChipSelect2 = 0; // Global variable for chip select 2
51 DWORD ChipSelect3 = 0; // Global variable for chip select 3
52 DWORD ChipSelect4 = 0; // Global variable for chip select 4
53 DWORD ChipSelect5 = 0; // Global variable for chip select 5
34,1 6%
```

Jflash.cpp에는 다음과 같이 jtag케이블 타입에 대해 처리하는 부분이 있다. HYBUS에서 사용하는 jtag은 패러럴 케이블을 바로 보드에 연결하는 형태인데 프린터 포트의 5번이 nTRST, 2번이 TCK, 3번이 TDI, 11번이 TDO, 4번이 TMS로 사용되고 있지만 일반적으로 현재 연구실에서 사용하는 jtag케이블과 다른 것 같다.

```
362 // Compare the selected cable name with all supported cables
363 if (!strcmp("INS", Cable_Type)){
364     CableType = Insight_Jtag;
365 } else if (!strcmp("PAR", Cable_Type)){
366     CableType = Parallel_Jtag;
367 }
```

케이블 타입에 따라서 출력하는 신호가 다르다.

```
938 if(CableType == Parallel_Jtag)
939 {
940     // TMS is D2, TDI is D1, and TCK is D0, so construct an output by creating a
941     // rising edge on TCK with TMS and TDI data set.
942     /* fixed */
943     _outp(lpt_address, tms*4+tdi*2+8); // TCK low
944     _outp(lpt_address, tms*4+tdi*2+1+8); // TCK high
945
946     // if we want to read the port, set TCK low because TDO is sampled on the
947     // TCK falling edge.
948     /* fixed */
949     if(rp == READ_PORT)
950         _outp(lpt_address, tms*4+tdi*2+8); // TCK low
951     if(rp == READ_PORT)
952         tdo = !((int)_inp(lpt_address + 1) >> 7); // get TDO data
953 }
954 else if (CableType == Insight_Jtag)
955 {
956     // There's some bit clearing here that isn't needed. It should make this
957     // code easier to understand.
958
959     //defines for the INSIGHT IJC-1 JTAG cable
960
961     /* the output port (lpt_address) */
962     #define INSIGHT_DIN 0x01
```

```

963     #define INSIGHT_CLK      0x02
964     #define INSIGHT_TMS_IN   0x04
965
966     /* Output Enable for the standard JTAG outputs
967     (not TDO since this is an output from the
968     chip we want to talk to */
969     #define nINSIGHT_CTRL    0x08
970     #define nINSIGHT_PROG    0x10 /* This causes the TDO line to be driven. We'll leave it high.*/
971
972
973     /*the input port (lpt_address + 1)*/
974     #define TDO_INPUT        0x10
975     #define TDO_INPUT_BITPOS 4
976
977     int lpt_data;
978
979     //form the data we want to write to the parallel port
980     lpt_data = nINSIGHT_PROG; //Output to TDO off
981     lpt_data &= ~nINSIGHT_CTRL; //Enable the outputs
982
983     if(tms == 1) lpt_data |= INSIGHT_TMS_IN;
984     if(tdi == 1) lpt_data |= INSIGHT_DIN;
985
986     // construct an output by creating a
987     // rising edge on TCK with TMS and TDI data set.
988     lpt_data &= ~INSIGHT_CLK;
989     _outp(lpt_address, lpt_data); // TCK low
990
991     lpt_data |= INSIGHT_CLK;
992     _outp(lpt_address, lpt_data); // TCK high
993
994     // if we want to read the port, set TCK low because TDO is sampled on the
995     // TCK falling edge.
996     if(rp == READ_PORT)
997     {
998         lpt_data &= ~INSIGHT_CLK;
999         _outp(lpt_address, lpt_data); // TCK high ??? Low?
1000         tdo = ((int)_inp(lpt_address + 1) & TDO_INPUT) >> TDO_INPUT_BITPOS; // get TDO data
1001     }
1002 }

```

다음과 유사하게 출력되어야 JFlashmm과 보드의 프로세서와 플래시 메모리가 동작한다는 것을 알 수 있다.

```

gurugio@gioserver: /home/SKKU/bulverde/jflashmm
[gurugio@gioserver jflashmm]$
[gurugio@gioserver jflashmm]$ su
Password:
[root@gioserver jflashmm]# jflashmm_skk bulbcx testfile.bin i

JFLASH Version 5.01.007
COPYRIGHT (C) 2000 - 2003 Intel Corporation

PLATFORM SELECTION:
Processor=      PXA27x
Development System= Mainstone
Data Version=   1.00.002

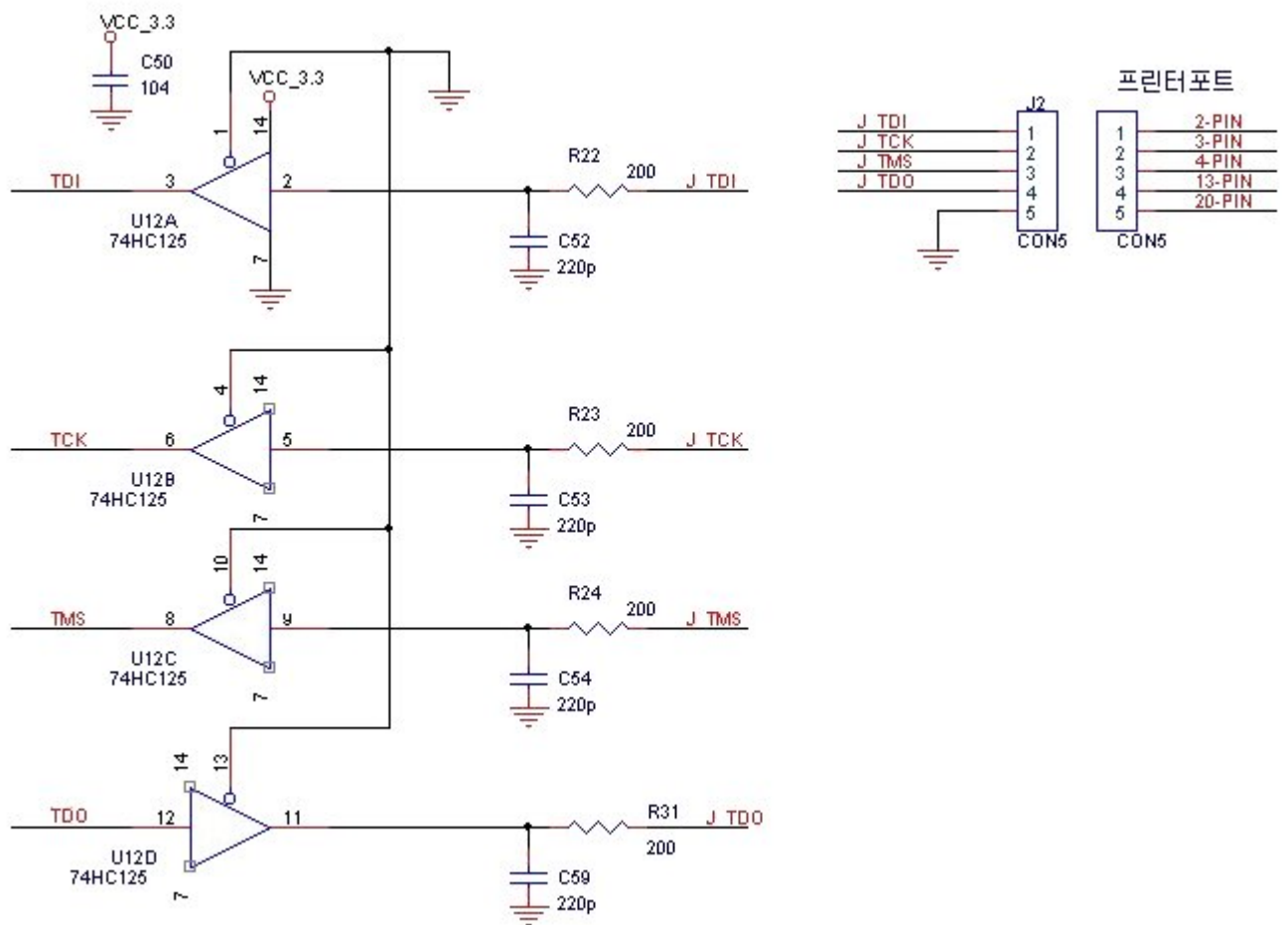
PXA27x revision C0
Found flash type: 28F128K3

Program successful completion.
Processor and Flash Memory Identified.
No programming operation performed.
[root@gioserver jflashmm]#

```


JTAG 동작을 테스트하는 방법

1. 프린터 포트를 이용하여 jtag을 사용할 때
 프린터 포트는 5V로 동작한다. jtag은 3V에서 동작하기 때문에 이것을 변환하는 회로가 필요하다. 다음은 삼성에서 만든 Secjtag의 회로도이다. PXA270의 JTAG 동작 전압은 3.3V이다. 하지만 PC의 프린터 포트의 전압은 5V이다. 이런 전압차가 프로세서의 데이터 버스와 주변 여러 회로에 노이즈를 발생시킨다. JTAG 케이블은 서로 다른 전압을 사용하는 신호를 변환해주면서 이런 노이즈에서 칩을 보호하는 역할을 한다. 보통 74HC245나 74HC125와 같은 단전원 칩을 사용하거나 3.3V와 5V의 양전원이 들어가는 74LV245를 사용하기도 한다. 노이즈를 최소한으로 줄이기 위해 각 라인을 저항으로 연결하고 캐패시터도 반드시 붙여준다.



회로를 살펴보고 jtag 케이블이 회로에 맞게 설계되었는지 전압이 올바르게 출력되는지를 확인한다.

2. TDI, TMS, TCLK의 신호가 발생하는가?
 예를 들어 보드를 켜지 않은 상태에서 jtag을 연결시키고 다음과 같은 명령을 실행하고,

jflashmm bulbcx testfile.bin i

오실로스코프를 1ms 단위 정도로 조정하고 TDI와 TCLK을 측정하면 아주 짧게라도 신호가 나올 것이다. 이정도 신호가 나온다면 일단 jtag 케이블은 동작이 된다고 판단해도 좋다.

3. Bulberde에서 nTRST에 high 신호가 인가되어 있는지 확인한다.
4. “jflashmm bulbcx testfile.bin i” 로 실행해서 identification이 되는지 확인한다.

JTAG 동작 확인 후 하드웨어 테스트

JTAG이 동작한다는 것은 프로세서와 플래시 메모리가 동작한다는 것을 뜻한다. 여기서 보다 확실하게 보드가 동작하는 지 또 JTAG이 프로그램을 정상적으로 퓨징할 수 있는지 확인하기 위해서 간단하게 LED를 깜박이고 시리얼에 문자를 출력하는 프로그램을 만들어 본다. 꼭 LED가 있어야 하는건 아니고 GPIO 하나를 골라서 주기적으로 신호를 보내서 오실로스코프로 파형이 발생하는지 확인하면 된다. Falinux의 EZ-X5 보드는 4개의 LED를 달아서 실행 도중 오류에 따라 다르게 LED를 깜박거린다. 따라서 어디에서 오류가 발생했는지 짐작할 수 있게 하였다. 보드를 설계할 때 이렇게 디버깅을 위한 고려도 하면 개발 시간을 아낄 수 있다.

가장 먼저 해야 할 일은 적당한 GPIO를 고르는 것이다. SKKU보드에는 따로 보드 동작을 위한 LED가 없으므로 임시로 현재 사용하지 않는 오디오 칩에 관련된 GPIO<30>을 사용한다.

PXA270의 GPIO는 입력과 출력으로 나뉘지고 입력과 출력 각각에 따라 3가지 기능이 있다. 즉 하나의 GPIO 핀이 6가지의 기능을 가지고 있는 셈이다. GPIO를 사용하기 전에 어떤 기능으로 사용해야 할지를 결정해서 입출력을 결정하는 레지스터와 기능을 결정하는 레지스터를 설정해주면 된다. Intel PXA27x Processor Family Developer's Manual에 24장 General-Purpose I/O Controller에서 Table 24-2를 보면 핀 이름과 기능들이 나열되어 있다. 회로에 따라 여기서 기능을 선택하여 GPIO 설정 레지스터의 값들을 결정하면 된다.

GPIO를 설정하는 레지스터는 GPLR, GPDR, GPSR, GPCR, GRER, GFER, GEDR, GAFR이 있다. Intel PXA27x Processor Family Developer's Manual에서 테이블 24-41을 보면 끝부분에 96번 이상의 GPIO를 설정하는 레지스터가 따로 있는 것을 알 수 있다. 이는 PXA270에서 PXA250부터 사용된 GPIO 이외에 추가됐기 때문이다. 각 레지스터를 간략하게 설명하면 다음과 같다.

- GPLR : 읽기 전용 상태 레지스터, High/Low 상태를 알 수 있다.
- GPDR : 입출력 방향 설정
- GPSR : Set, 하이 신호 인가
- GPCR : Clear, 로우 신호 인가
- GRER/GFER : Rising Edge 디텍트, Falling Edge 디텍트. GEDR에 디텍트된 결과 값이 출력된다. 만약 Rising Edge 디텍트 기능을 사용한다면 해당 핀에 Rising edge가 발생할 때마다 GEDR의 해당 비트가 1로 셋팅되고 이 비트에 1을 쓰면 0으로 클리어 하게 된다.
- GEDR : GRER/GFER에 설정한대로 Edge를 디텍트하면 1로 설정되고 1을 써주어서 셋팅된 값을 클리어 한다.
- GAFR : 하나의 GPIO마다 2비트씩 설정할 수 있다. 0은 GPIO로 사용하는 것이고 1~3은 테이블 24-2에 나온 기능들을 사용하도록 설정하는 것이다. 회로도에 따라 어떤 기능을 사용하는지 세심하게 결정해서 이 값을 결정해야 한다.

예를 들어 GPIO<30>에 관련된 설명을 보면 입력으로는 따로 사용되는 기능이 없고 출력으로는 I2S_SDATA_OUT, AC97_SDATA_OUT, USB_P3_2에 사용된다고 써있다. GAFR0_L 레지스터에서 28,29번 비트의 설정 값을 00으로 해주면 이런 기능들이 아니라 GPIO로 사용된다. 그리고 GPDR0의 30번 비트를 1로 셋팅해서 출력으로 설정해주면 된다. GPSR0의 30번 비트에 1을 써서 하이 값을 출력하고 잠시 후에 GPCR0의 30번 비트에 1을 써서 로우 값을 출력하고 잠시 후에 다시 하이 값을 출력하는 일을 반복하면 된다. 현재 보드에는 오디오에 관련된 칩을 부착하지 않고 오직 프로세서와 메모리, 전원관련 칩들만 부착했으므로 신경쓰지 말고 오실로스코프로 오디오 칩의 AC97_SDATA_OUT 핀을 찍어보면서 프로그램 동작을 확인하면 된다.

FFUART를 사용하기 위해서 테이블 24-2와 SKKU 보드의 회로도를 보면 GPIO<34> ~

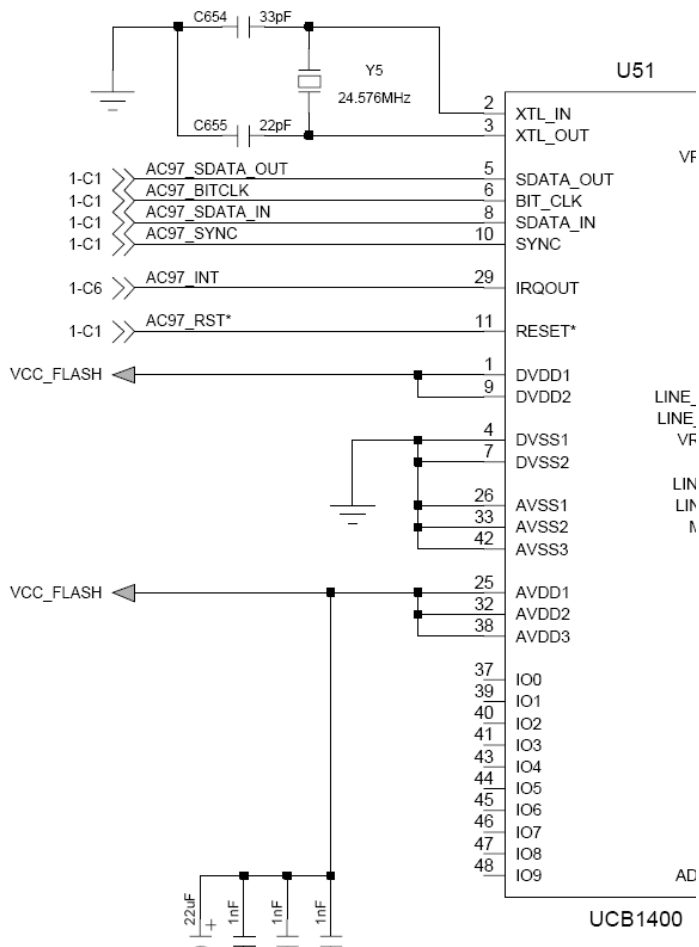
GPIO<41>을 사용하는 것을 알 수 있다. GPIO<34> ~ GPIO<38>은 입력으로, GPIO<39> ~ GPIO<41>은 출력으로 사용된다. 각 핀의 사용법은 다음과 같다.

- GPIO<34> : FFRXD
- GPIO<35> : FFCTS
- GPIO<36> : FFDCD
- GPIO<37> : FFDSR
- GPIO<38> : FFRI
- GPIO<39> : FFTXD
- GPIO<40> : FFDTR
- GPIO<41> : FFRTS

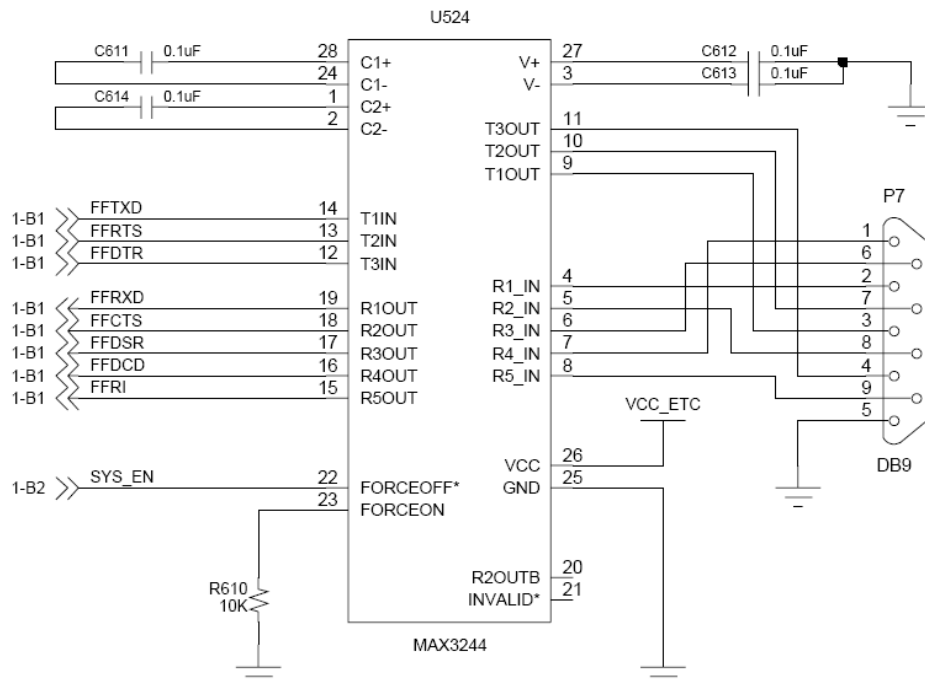
PXA270의 UART에는 SUART(Standard UART)와 FFUART(Full Function UART)가 있다. 이름에서 알 수 있듯이 SUART에는 TxD와 RxD만이 있다. Mainstone 보드에는 FFUART를 사용하므로 SKKU에도 동일하게 구성했다.

이후의 프로그램 소스와 컴파일 과정 등은 마이크로소프트 2003년 10월부터 유명창님께서 연재하신 ‘ARM 부트로더 제작기’를 참조하여 만들었다. gcc 버전에 따라 달라진 내용과 우리 보드에 맞게 수정한 내용을 주로 설명한다.

다음 그림은 오디오 칩 회로도의 일부를 나타낸 것이다. AC97_SDATA_OUT이 5번 핀으로 되어있다.



다음 그림은 시리얼 통신을 위한 회로이다.



로 값을 설정한다.

```
24 #define DEBUG_LED      (1<<30)
25
```

u-boot 1.1.2 소스에서 복사한 FFUART 설정 레지스터

```
26 /* Full Function UART (FFUART) */
27 #define FFUART  0x40100000
28 #define FFRBR 0x40100000 /* Receive Buffer Register (read only) */
29 #define FFTHR 0x40100000 /* Transmit Holding Register (write only) */
30 #define FFIER 0x40100004 /* Interrupt Enable Register (read/write) */
31 #define FFIR 0x40100008 /* Interrupt ID Register (read only) */
32 #define FFFCR 0x40100008 /* FIFO Control Register (write only) */
33 #define FFPCR 0x4010000C /* Line Control Register (read/write) */
34 #define FFMCR 0x40100010 /* Modem Control Register (read/write) */
35 #define FFISR 0x40100014 /* Line Status Register (read only) */
36 #define FFMSR 0x40100018 /* Modem Status Register (read only) */
37 #define FFSPR 0x4010001C /* Scratch Pad Register (read/write) */
38 #define FFISR 0x40100020 /* Infrared Selection Register (read/write) */
39 #define FFDLL 0x40100000 /* Divisor Latch Low Register (DLAB = 1) (read/write) */
40 #define FFDLH 0x40100004 /* Divisor Latch High Register (DLAB = 1) (read/write) */
41
```

클럭 설정 레지스터. FFUART에 들어가는 클럭을 활성화시킨다.

```
42 #define CKEN 0x41300004 /* Clock Enable Register */
43 #define CKEN6_FFUART (1<<6) /* CKEN[6] = FFUART unit clock enable
44
```

FFPCR의 설정 값

```
45 #define LCR_WLS0 (1) /*
46 #define LCR_WLS1 (1 << 1)
47 #define LCR_DLAB (1 << 7)
48
```

기타 FFUART 설정값

Baudrate는 38400bps로 하고 8-N-1로 설정한다.

```
49 #define QUOT 24 /* 38400 */
50 #define IER_UUE (1 << 6)
51
```

FFUART를 사용하기 위한 GPIO 설정 값. 반드시 한 비트씩 확인해서 이해할 것

```
52 #define CFG_GPCR1_VAL 0x00000380
53 #define CFG_GPCR1_VAL 0x00000000
54 #define CFG_GPCR1_VAL 0x00000380
55 #define CFG_GAFR1_L_VAL 0x0000a950
56
57 .text
58
59 .global _start
60
61 _start:
62
63     b reset
64     b undefined_instruction
65     b software_interrupt
66     b prefetch_abort
67     b data_abort
68     b not_used
69     b IRQ
70     b FIQ
71
72
73 reset:
74
```

GPIO<30>를 출력으로 지정

```
76     ldr r1, =DEBUG_LED
77
78     // set direction of 4th bit as output
```

```

79  ldr r0, =GPDR0 // r0 = PXA_REG_GP_BASE
80  str r1, [r0] // [r0+GPDR0] = r1
81

```

로우 신호 인가

```

82  // set 4th bit as low -> turn on LED
83  ldr r0, =GPCR0
84  str r1, [r0] // [r0+GPCR0] = r1
85
86

```

FFUART를 위한 GPIO 설정

```

87 GPIO_init:
88  /* GPIO<34-41> are used for FFUART */
89  /* GPIO<34-41> are controlled by GPLR1, GPSR1, GPCR1, CPDR1,
90     GRER1, GFER1, GEDR1, GAFR1_L */
91  /* GPIO<34-38> are used as INPUT, Function 1 */
92  /* GPIO<39-41> are used as OUTPUT, Function 2 */
93  /* GPDR1 = 0x380 */
94  /* GPSR1 = 0x0 */
95  /* GPCR1 = 0x380 */
96  /* GRER1,GFER1,GPLR1 = don't care */
97  /* GAFR1_L = 0xa950 */
98  ldr r0, =GPSR1
99  ldr r1, =CFG_GPSR1_VAL
100 str r1, [r0]
101
102 ldr r0, =GPCR1
103 ldr r1, =CFG_GPCR1_VAL
104 str r1, [r0]
105
106 ldr r0, =GPDR1
107 ldr r1, =CFG_GPDR1_VAL
108 str r1, [r0]
109
110 ldr r0, =GAFR1_L
111 ldr r1, =CFG_GAFR1_L_VAL
112 str r1, [r0]
113

```

FFUART를 위한 초기화 과정

순서를 지켜서 실행해야 함

```

114 serial_init:
115  /* CKEN |= CKEN6_FFUART */
116  /* Enable FFUART clock */
117  ldr r0, =CKEN
118  ldr r1, [r0]
119  ldr r2, =CKEN6_FFUART
120  orr r1, r1, r2
121  str r1, [r0]
122
123  /* FFIER = FFFCR = 0 */
124  /* Disable interrupts and FIFOs */
125  mov r0, #0
126  ldr r1, =FFIER
127  ldr r2, =FFFCR
128  str r0, [r1]
129  str r0, [r2]
130
131  /* FFLCR = LCR_WLS0 | LCR_WLS1 | LCR_DLAB */
132  /* 8bit character, Access Divisor Latch registers (DLL, DLH) */
133  mov r1, #LCR_WLS0
134  orr r1, r1, #LCR_WLS1
135  orr r1, r1, #LCR_DLAB
136  ldr r0, =FFLCR
137  str r1, [r0]
138
139  /*FFDLL = QUOT & 0xff, low byte of baudrate */

```

```

140  mov r1, #QUOT
141  ldr r0, =FFDLL
142  str r1, [r0]
143
144  /* FFDLH = QUOT >> 8, high byte of baudrate */
145  mov r1, #0
146  ldr r0, =FFDLH
147  str r1, [r0]
148
149  /* FFLCR = LCR_WLS0 | LCR_WLS1 */ 150 /* Clear DLAB bit */
151  mov r1, #LCR_WLS0
152  orr r1, r1, #LCR_WLS1
153  ldr r0, =FFLCR
154  str r1, [r0]
155
156

```

FFUART 동작 시작

```

157  /* FFIER = IER_UUE */
158  /* Enable UART unit */
159  mov r1, #IER_UUE
160  ldr r0, =FFIER
161  str r1, [r0]
162

```

오퍼랜드 범위가 12비트를 넘는다. ARM에서 오퍼랜드의 크기는 12비트가 한계이므로 mov 명령어가 아니라 ldr 명령어를 사용한다.

```

163  ldr r1, =DEBUG_LED
164

```

일정 시간 딜레이

```

165 B10:
166  mov r4, #WAIT_TIME_NOCACHE // r4 = WAIT..
167 B20:
168  nop
169  nop
170  subs r4, r4, #1 // r4 = r4 - 1 & effect CPSR
171  bne B20 // if NE(not equal) is set (r4 != 0), branch to B20
172

```

GPIO<30>에 로우 신호 인가

```

173  // set 4th bit as high -> turn off LED
174  ldr r0, =GPSR0
175  str r1, [r0] // [r0+GPSR0] = r1
176
177  mov r4, #WAIT_TIME_NOCACHE // r4 = WAIT_TIME_NOCACHE
178
179 B30:
180  nop
181  nop
182  subs r4, r4, #1
183  bne B30
184

```

GPIO<30>에 하이 신호 인가

```

185  // turn on LED
186  ldr r0, =GPCR0
187  str r1, [r0]
188

```

시리얼에 '0' 출력

```

189  // serial_putc(0)
190  mov r2, #0x30
191  ldr r3, =FFTHR
192  str r2, [r3]
193
194  b B10
195
196 error_loop:

```



```

197
198 undefined_instruction:
199 software_interrupt:
200 prefetch_abort:
201 data_abort:
202 not_used:
203 IRQ:
204 FIQ:
205
206     b error_loop
207
208 // Here is end of program

```

다음은 로더를 위한 스크립트 파일 `led-ld-script`이다. 어셈블을 하고 코드의 시작 주소를 지정해 주기 위해서 필요하다. ARM은 리셋 직후 0x0번지의 명령어를 실행하므로 프로그램 코드의 시작 주소가 0x0이 되도록 로더를 설정해야 한다.

```

1 OUTPUT_FORMAT("elf32-littlearm", "elf32-littlearm", "elf32-littlearm")
2 OUTPUT_ARCH(arm)
3 ENTRY(_start)
4 SECTIONS
5 {
6     . = 0x00000000;
7
8     . = ALIGN(4);
9     .text : { *(.text) }
10
11    . = ALIGN(4);
12    .text : { *(.rodata) }
13
14    . = ALIGN(4);
15    .text : { *(.data) }
16
17    . = ALIGN(4);
18    .text : { *(.got) }
19
20    . = ALIGN(4);
21    .text : { *(.bss) }
22 }

```

다음은 Makefile이다.

```

1
2 CC = arm-linux-gcc
3 LD = arm-linux-ld
4 OC = arm-linux-objcopy
5
6 INCLUDE = -nostdinc -I. -I$(TOPDIR)/include
7
8 CFLAGS = $(INCLUDE)
9 CFLAGS += -Wall -Wstrict-prototypes -Wno-trigraphs -O2
10 CFLAGS += -fno-strict-aliasing -fno-common -pipe -mapcs-32

```

gcc 버전이 3이 넘어가면서 바뀐 옵션이 있다. `short-load-bytes` 옵션은 이름이 `alignment-traps`로 바뀌고 디폴트로 설정된다. 또 `-Wa,mxscale` 옵션은 필요없다. PXA27x는 armv5 명령어 셋을 사용하므로 다음과 같은 옵션들이 필요하다.

```

11 #CFLAGS += -march=armv5 -Wa,-mxscale -mtune=xscale -mshort-load-bytes -msoft-float -fno-builtin
12 # in gcc-3.x, short-load-bytes is renamed to alignment-traps and seted as default
13 # -Wa,-mxscale option should be removed for gcc 3.x
14 CFLAGS += -march=armv5 -mtune=xscale -msoft-float -fno-builtin
15
16 START_LDFLAGS = -p -X -T ./led-ld-script
17

```

```

18 OCFLAGS = -O binary -R .note -R .comment -S
19
20 BOOT_IMAGE = ledtest_pxa270
21
22 SRCS = led.S
23 OBJS = led.o
24
25 TARGET = ledtest_org
26 PRE_TARGET = ledtest-elf32
27
28 %.o:%.S
29     @echo "Assembler compiling $< ..."
30     $(CC) -c $(CFLAGS) -o $@ $<
31
32 #make image file
33
34 all:$(PRE_TARGET)
35     $(OC) $(OCFLAGS) $(PRE_TARGET) $(TARGET)
36     dd if=$(TARGET) of=$(BOOT_IMAGE) bs=1k conv=sync
37
38 $(PRE_TARGET):$(OBJS)
39     $(LD) $(START_LDFLAGS) -o $@ $(OBJS)
40
41 clean:
42     rm -f *.o
43     rm -f $(PRE_TARGET)
44     rm -f $(TARGET)
45

```

혹시 어셈블을 할 때 old and new-style option 이라는 에러가 나오면 새로운 버전의 gcc에 맞는 옵션이 아니라는 것이므로 옵션을 수정해야 한다. google에서 설정한 옵션들을 검색해보면 같은 에러에 관해 질문한 내용들이 많을 것이다. 찾아서 답변들을 읽고 고치거나 삭제하면 된다.

이제 다음과 같이 퓨징을 한다.

```
gurugio@gioserver: /home/SKKU/bulverde/jflashmm
Flash_8803_2_32.dat Flash_8813_2_32.dat jflashmm25x32 pxa27x16.dat
Flash_8806_2_32.dat Flash_8814_2_32.dat jflashmm27x16 pxa27x32.dat
Flash_8807_2_32.dat Flash_8815_1_16.dat jflashmm27x32 RelNote_JFlashmm.htm
Flash_8808_2_32.dat Flash_8815_2_32.dat jflashmm.dsp SWLicense.pdf
Flash_880C_1_16.dat Flash_8816_2_32.dat jflashmm.dsw testfile.bin
Flash_880C_2_32.dat giveio.inf jflashmm.mak
Flash_880D_1_16.dat giveio.ini jflashmm.ncb
Flash_880D_2_32.dat giveio.sys jflashmm_skku
[root@gioserver jflashmm]# jflashmm bulbcx ledtest_pxa270 p

JFLASH Version 5.01.007
COPYRIGHT (C) 2000 - 2003 Intel Corporation

PLATFORM SELECTION:
Processor= PXA27x
Development System= Mainstone
Data Version= 1.00.002

PXA27x revision C0
Found flash type: 28F128K3
The last 90 percent of image file is all zeros
Would you like to save time by not programming that area? [y/n]: y

Unlocking block at address 0
Erasing block at address 0
Starting programming
Using BUFFER programming mode...
Programming done
Starting Verify
verify error at address = 0 exp_dat = ea000006 act_dat = a0000006

[root@gioserver jflashmm]#
[root@gioserver jflashmm]#
```

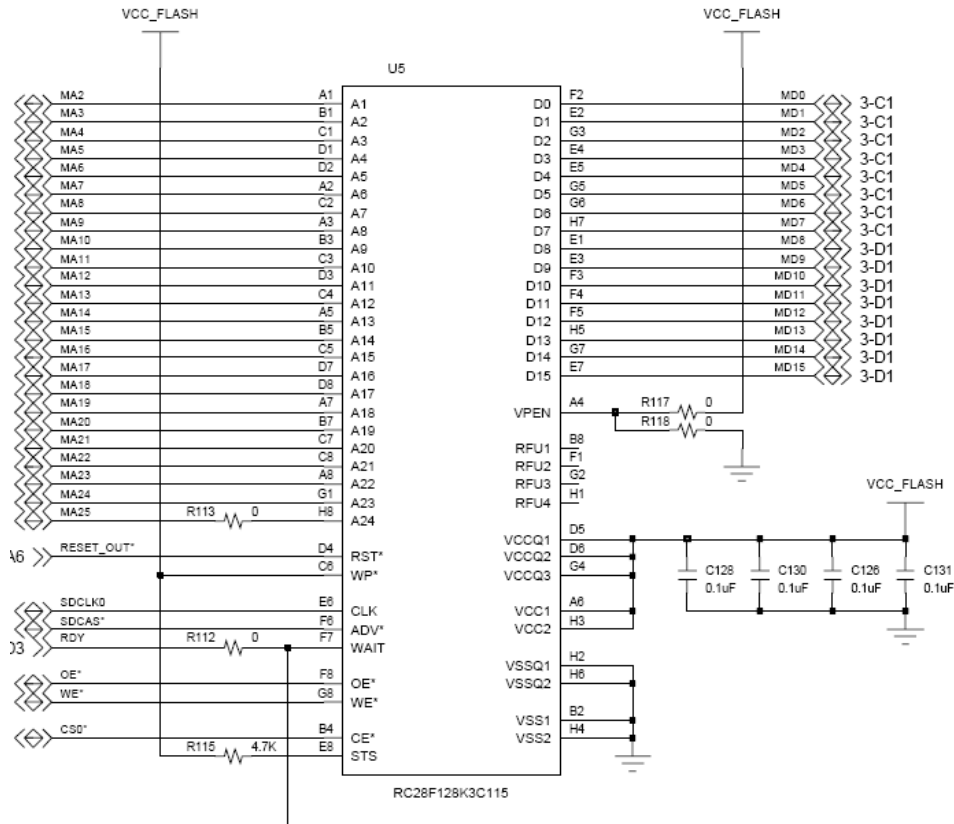
테스트 프로그램을 다운로드 한 후 보드를 리셋하고 이 핀을 확인해서 신호가 출력되고 시리얼을 통해서 문자도 출력된다면 jflash도 정상적으로 동작하고 보드도 정상적이라는 것을 알 수 있다. 이제 최소한의 하드웨어 상태를 알았으니 부트 로더를 포팅할 준비가 끝났다.

메모리관련 회로 이해

SKKU의 보드에서 부트 로더에 필요한 사항들을 살펴본다.

먼저 메모리에 대해서 살펴보면 SKKU 보드에는 2개의 플래시 롬과 2개의 SDRAM이 있다. 메모리 컨트롤에 관한 사항은 Intel PXA27x Processor Family Developer's Manual에서 chap 6 Memory Controller를 참조한다.

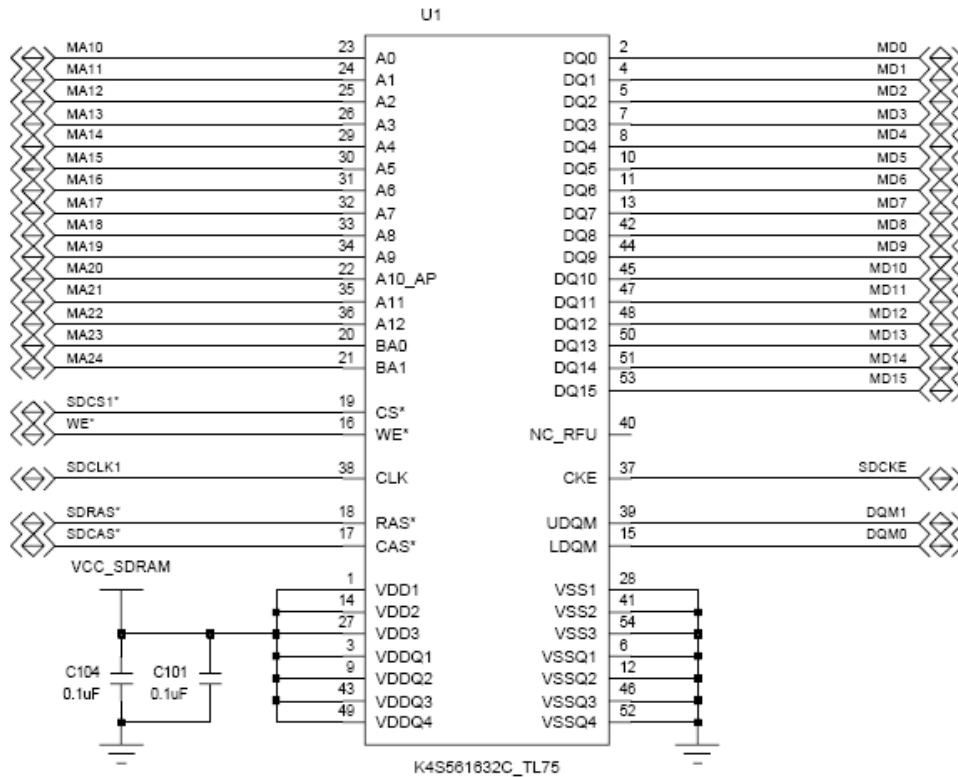
플래시 롬은 인텔의 Strata Flash 28F128K3를 사용한다. 이 플래시 메모리는 25ns 시간이 소모되는 비동기식 접근과 13ns 시간이 소모되는 동기식 접근이 지원된다. 흔히 사용하는 Strata Flash 28F128J3A는 비동기식 접근만 지원되는데 반해 이 플래시는 좀더 빠른 동기식 접근과 버스트 모드를 지원한다. 모델 번호에서 알 수 있듯이 128Mbit의 크기를 가지며 128블록으로 구성되어 있어서 한 섹터당 크기가 1Mbit, 즉 256Kbyte가 되고 NOR 플래시 이다. K3는 3volt 전원이 필요하다는 것을 나타낸다. 칩 하나의 크기가 128Mbit, 16Mbyte이고 두 개를 사용하므로 32Mbyte의 플래시 롬을 가지게 된다. 다음은 SKKU 보드의 플래시 회로이다.



하나의 28F128K3은 8M 개의 워드로 구성되므로 23개의 주소 핀이 필요하다. 28F128K3의 매뉴얼을 보면 128Mbit의 칩에는 최상위 주소 버스 라인이 A23에 연결하도록 했다. 그래서 R113을 없애고 A24를 연결하지 않는다. 버스 폭이 32비트이므로 MA<1:0>은 주소 지정에 필요 없으므로 MA<24:2>를 사용한다. SCDLK0을 클럭으로 사용하고 nCS0으로 칩 선택을 하므로 플래시 롬의 시작 주소는 0x00000000이 된다. 즉 롬을 위한 뱅크 중에서 첫번째 뱅크로 지정된다. SKKU 보드에서는 비동기로 사용하므로 WAIT를 연결하지 않고 STS에 하이 신호를 인가한다. 또 VPEN에 하이 신호를 인가해서 데이터를 프로그래밍할 수 있도록 한다. 각 핀의 역할은 28F128K3의 칩 매뉴얼을 참고한다.

SDRAM은 삼성의 K4S561632C-TL75를 사용한다. 하나의 칩이 4Mbit X 16bit X 4banks로 구

성되어 있어서 크기가 256Mbit, 즉 32MByte가 된다. 하나의 칩의 데이터 폭이 16비트이고 4개의 뱅크로 이루어져있다. 매뉴얼을 확인하면 TL75가 최대 133MHz로 동작할 수 있다는 것을 나타낸다고 알 수 있다. 매뉴얼에서 확인해야 할 사항들이 CAS latency가 2나 3이고 64ms의 리프레시 주기를 가지며 CL=3라는 것을 알아두어야 한다. 다음은 SKKU 보드의 회로도중 SDRAM에 관한 부분이다.



칩 셀렉트 핀인 nCS에는 SDCS1을 연결했다. SDCS0이 아니라 SDCS1을 연결한 특별한 이유는 없다. 어쨌든 SDCS1을 연결했으므로 SDRAM의 시작 주소는 0xA4000000이 된다. PXA270의 SDCS0~3 핀은 SDRAM 파티션을 선택하기 위한 핀이다. SDCS0은 물리 주소 0xA0000000에서 시작하고 한 파티션의 크기는 64MByte이다. 따라서 다음 파티션 SDCS1은 0xA4000000이고 각파티션은 0x04000000씩 떨어져있다(Intel PXA27x Processor Family Developer's Manual에서 Figure 6-2 참조). 클럭은 SDCLK1이 연결되어 있고 주소 라인이 10번 핀부터 24번 핀이 연결되어 있다. 주소 라인의 연결은 Intel PXA27x Processor Family Developer's Manual에서 테이블 6-3을 참조한다. 이 테이블은 일반적인 주소 지정 방식을 사용하고 SDCLK<0>과 SDCLK<3>이 플래시에 사용되지 않을 때, 즉 가장 일반적으로 사용되는 SDCLK가 SDRAM에만 사용되고 nCS가 플래시 롬에 사용되는 상황에서의 주소 라인 사용에 대해서 나열되어 있다. 보드에 사용된 SDRAM의 매뉴얼에서 A0~A12에 관한 내용을 찾아보면 ROW 비트가 RA0~RA12로 13개, COLUMN 비트가 CA0~CA8로 9개, 데이터 폭은 DQ0~DQ15로 16비트, 뱅크 선택은 BA0~BA1로 2비트로 구성되어 있다는 것을 알 수 있다. 테이블 6-3에서 이런 특성을 찾아보면 테이블 가장 위에 MA<24:10>이 사용되어야 한다고 설명되어 있고 MDCNFG라는 레지스터에서 STACK 항목의 값이 00이어야 한다고 설명하고 있다. MDCNFG 레지스터에 대한 내용은 부트 로더 설정에서 살펴본다. 테이블에서 진하게 출력

된 핀 번호 MA<24:23>이 뱅크 설정을 위한 BA0~BA1 핀이다. 즉 MA<24>를 BA1에 MA<32>을 BA0에 연결해야 한다. 보다 자세한 주소 지정 방식은 Intel PXA27x Processor Family Developer's Manual에서 6.4.2.3 SDRAM Memory Size Options을 참조한다. 현재 대부분의 PXA270, PXA25x보드가 삼성의 램을 사용하므로 비슷한 설정을 사용한다. 다른 보드들에 대한 자료를 찾아서 비교해보면 더 잘 이해할 수 있다. 그 외의 핀들에 대한 설명은 SDRAM의 매뉴얼과 Intel PXA27x Processor Family Developer's Manual의 chap 6을 참조한다. 반드시 각 핀에 대한 역할을 이해해야만 부트 로더 설정을 이해하고 필요한 값을 찾을 수 있다.

부트로더에 설정 파일 추가하기

u-boot를 사용하기 위해서는 u-boot와 u-boot를 컴파일하기 위한 크로스 컴파일러 패키지 ELDK를 준비해야 한다. ELDK에 대한 자세한 사항은 <http://www.denx.de/wiki/DULG/ELDK>를 참조한다.

다음은 부트로더 소스와 크로스 컴파일러를 다운 받을 수 있는 링크 주소이다.

u-boot : <http://sourceforge.net/projects/u-boot/>

ELDK : <http://www.denx.de/wiki/view/DULG/ELDKAvailability>

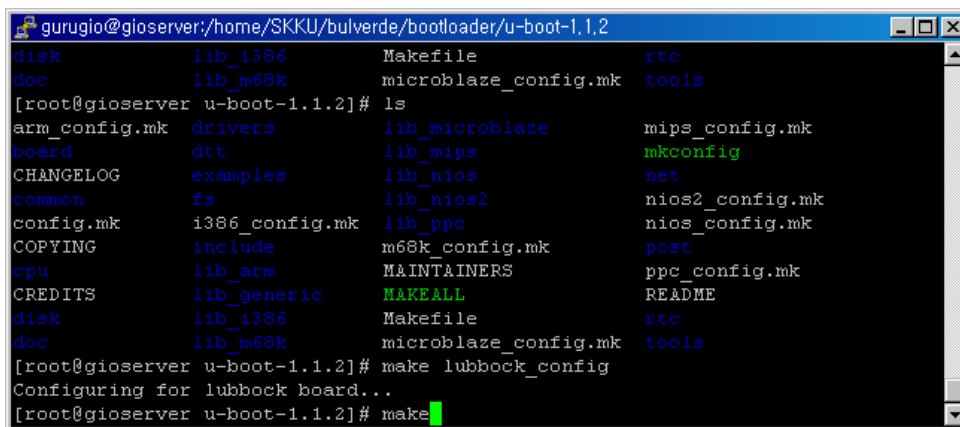
몇몇 버전이 다른 크로스 컴파일러의 arm-linux-ld를 사용하면 'libgcc.a uses VFP instructions, whereas u-boot does not'이라는 에러가 생기면서 컴파일을 할 수 없을 때가 있다. 따라서 가급적이면 ELDK를 사용하도록 한다. ELDK 사이트에 들어가서 arm버전 시디 이미지를 다운받는다. mount -o loop arm-버전.iso /media/cdrom 와 같이 mount 명령어로 마운트하고 디렉토리에 들어가서 ./install -d /usr/local/ELDK 명령어로 설치한다. 간단하게 설치할 수 있어서 편리하고 2.6 커널을 지원한다.

u-boot는 1.1.3 버전까지 개발됐고 우리는 1.1.2 버전을 사용한다.

먼저 컴파일이 잘 되는지 확인하기 위한 테스트로 lubbock를 위한 설정과 컴파일을 해본다. 다음 그림과 같이 먼저 원하는 보드 타입에 대한 설정을 해주고 그 다음에 컴파일을 실행하면 된다.

```
make skku_config
```

```
make
```



```
gurugio@gioserver:/home/SKKU/bulverde/bootloader/u-boot-1.1.2
disk      lib_i386      Makefile      rtc
doc        lib_m68k      microblaze_config.mk  tools
[root@gioserver u-boot-1.1.2]# ls
arm_config.mk  drivers      lib_microblaze  mips_config.mk
board          dtc          lib_mips        mkconfig
CHANGELOG      examples    lib_nios        net
common         fs          lib_nios2       nios2_config.mk
config.mk      i386_config.mk lib_ppc         nios_config.mk
COPYING        include     m68k_config.mk  post
cpu            lib_arm     MAINTAINERS     ppc_config.mk
CREDITS        lib_generic MAKEALL         README
disk           lib_i386    Makefile        rtc
doc            lib_m68k    microblaze_config.mk  tools
[root@gioserver u-boot-1.1.2]# make lubbock_config
Configuring for lubbock board...
[root@gioserver u-boot-1.1.2]# make
```

그 외에 사용되는 명령어는 make clobber가 있는데 이는 보드에 대한 설정 파일들까지 삭제하게 된다. make clean은 오브젝트 파일들과 컴파일러가 생성한 중간 파일들을 지운다.

다음 그림과 같이 u-boot.bin 파일이 생성되면 컴파일이 정상적으로 완료된 것이다. 이 u-boot.bin 파일이 플래시 롬에 퓨징될 바이너리 코드이다. 이렇게 바이너리 코드가 이상없이 생성된다면 크로스 컴파일러가 정상적으로 설치된 것이므로 우리 보드에 대한 설정을 시작한다.

```
gurugio@gioserver:/home/SKKU/bulverde/bootloader/u-boot-1.1.2
ock.a cpu/pxa/libpxa.a lib_arm/libarm.a fs/cramfs/libcramfs.a fs/fat/libfat.a
fs/fdos/libfdos.a fs/jffs2/libjffs2.a fs/reiserfs/libreiserfs.a fs/ext2/libext
2fs.a net/libnet.a disk/libdisk.a rtc/librtc.a dtb/libdtb.a drivers/libdrivers
.a drivers/sk98lin/libsk98lin.a post/libpost.a post/cpu/libcpu.a common/libcom
mon.a |sed -n -e 's/.*\(_u_boot_cmd_.*\)/-u\1/p'|sort|uniq';\
arm-linux-ld -Bstatic -T /home/SKKU/bulverde/bootloader/u-boot-1.1.2/b
oard/lubbock/u-boot.lds -Ttext 0xa3080000 $UNDEF_SYM cpu/pxa/start.o \
--start-group lib_generic/libgeneric.a board/lubbock/liblubbo
ck.a cpu/pxa/libpxa.a lib_arm/libarm.a fs/cramfs/libcramfs.a fs/fat/libfat.a fs
/fdos/libfdos.a fs/jffs2/libjffs2.a fs/reiserfs/libreiserfs.a fs/ext2/libext2f
s.a net/libnet.a disk/libdisk.a rtc/librtc.a dtb/libdtb.a drivers/libdrivers.a
drivers/sk98lin/libsk98lin.a post/libpost.a post/cpu/libcpu.a common/libcommo
n.a --no-warn-mismatch -L /usr/local/ELDK/usr/bin/../../lib/gcc-lib/arm-linux/3.3
.3 -lgcc --end-group \
-Map u-boot.map -o u-boot
arm-linux-objcopy --gap-fill=0xff -O srec u-boot u-boot.srec
arm-linux-objcopy --gap-fill=0xff -O binary u-boot u-boot.bin
[root@gioserver u-boot-1.1.2]#
```

다음은 우리 보드에 맞는 설정을 추가해주는 것이다. 다음 과정을 순서대로 실행하면 된다.

1. Makefile 수정

Makefile을 열면 다음과 같은 부분이 있다.

```
gurugio@gioserver:/home/SKKU/bulverde/bootloader/u-boot-1.1.2
41
42 TOPDIR := $(shell if [ "$$PWD" != "" ]; then echo $$PWD; else pwd; fi)
43 export TOPDIR
44
45 ifeq (include/config.mk,$(wildcard include/config.mk))
46 # load ARCH, BOARD, and CPU configuration
47 include include/config.mk
48 export ARCH CPU BOARD VENDOR SOC
49 # load other configuration
50 include $(TOPDIR)/config.mk
51
52 ifndef CROSS_COMPILE
53 ifeq ($(HOSTARCH),ppc)
54 CROSS_COMPILE =
55 else
56 ifeq ($(ARCH),ppc)
57 CROSS_COMPILE = ppc_8xx-
58 endif
59 ifeq ($(ARCH),arm)
60 CROSS_COMPILE = arm-linux-
61 endif
62 ifeq ($(ARCH),i386)
63 ifeq ($(HOSTARCH),i386)
"Makefile" 1596L, 48636C 52,1 2%
```

CROSS_COMPILE이 설정되지 않은 상태이다. 따라서 현재 설치된 크로스 컴파일러의 경로를 지정해준다.


```
gurugio@gioserver:/home/SKKU/bulverde/bootloader/u-boot-1.1.2_skku
42 TOPDIR := $(shell if [ "$$PWD" != "" ]; then echo $$PWD; else pwd; fi)
43 export TOPDIR
44
45 ifeq (include/config.mk,$(wildcard include/config.mk))
46 # load ARCH, BOARD, and CPU configuration
47 include include/config.mk
48 export ARCH CPU BOARD VENDOR SOC
49 # load other configuration
50 include $(TOPDIR)/config.mk
51
52 CROSS_COMPILE = arm-linux-
53
54 ifndef CROSS_COMPILE
55 ifeq ($(HOSTARCH),ppc)
56 CROSS_COMPILE =
57 else
58 ifeq ($(ARCH),ppc)
59 CROSS_COMPILE = ppc_8xx-
60 endif
61 ifeq ($(ARCH),arm)
62 CROSS_COMPILE = arm-linux-
63 endif
64 ifeq ($(ARCH),i386)
```

그리고 필요없는 example 부분을 삭제한다.

```
gurugio@gioserver:/home/SKKU/bulverde/bootloader/u-boot-1.1.2
125
126
127 # The "tools" are needed early, so put this first
128 # Don't include stuff already done in $(LIBS)
129 SUBDIRS = tools \
130     examples \
131     post \
132     post/cpu
133 .PHONY : $(SUBDIRS)
134
135 #####
136 #####
137 #####
```

```
gurugio@gioserver:/home/SKKU/bulverde/bootloader/u-boot-1.1.2
125
126
127 # The "tools" are needed early, so put this first
128 # Don't include stuff already done in $(LIBS)
129 SUBDIRS = tools \
130     post \
131     post/cpu
132 .PHONY : $(SUBDIRS)
133
134 #####
135 #####
136 #####
```

그 다음은 SKKU 보드에 대한 컴파일 설정을 추가한다. PXA250 프로세서를 사용하는 보드들에 대한 설정이 있는데 그 밑에 PXA270 프로세서를 사용하는 SKKU 보드에 대한 설정을 추가한

다. arm pxa270 skku라고 된 구문은 lib_arm/ 디렉토리를 컴파일하고 cpu/pxa270/ 을 컴파일 하고 skku_config를 생성하도록 하는 문장이다. cpu/pxa270 디렉토리는 직접 만들어줘야 한다. cpu/pxa 를 복사해서 만들고 수정해서 쓴다.

```

gurgio@gioserver:/home/SKKU/bulverde/bootloader/u-boot-1.1.2
1390  @./mkconfig ${@:_config=} arm pxa wep250
1391
1392 xaenias_config : unconfig
1393  @./mkconfig ${@:_config=} arm pxa xaenias
1394
1395 xm250_config : unconfig
1396  @./mkconfig ${@:_config=} arm pxa xm250
1397
1398 xsengine_config : unconfig
1399  @./mkconfig ${@:_config=} arm pxa xsengine
1400
1401 #####
1402 ## Bulverde Systems
1403 #####
1404
1405 skku_config : unconfig
1406  @./mkconfig ${@:_config=} arm pxa270 skku
1407
1408 #=====
1409 # i386
1410 #=====
1411 #####
1412 ## AMD SC520 CDP
1413 #####
1414 sc520_cdp_config : unconfig
1415  @./mkconfig ${@:_config=} i386 i386 sc520_cdp
1416
1417 sc520_spunk_config : unconfig
1418  @./mkconfig ${@:_config=} i386 i386 sc520_spunk
"Makefile" 1604L, 48886C written
1402,11 88%

```

2. cpu/pxa270/ 디렉토리 추가

cpu/pxa 디렉토리를 복사해서 pxa270 디렉토리를 만든다. cpu/pxa 디렉토리는 PXA250을 위한 코드들을 가지고 있으므로 PXA270에 맞게 수정해야 한다.

```

gurgio@gioserver:/home/SKKU/bulverde/bootloader/u-boot-1.1.2/cpu
[root@gioserver cpu]# ls
74xx_7xx  arm926ejs  lh7a40x  mpc5xx  mpc8260  nios2  sa1100
arm720t  at91rm9200  mcf52x2  mpc5xxx  mpc85xx  ppc4xx
arm920t  i386  microblaze  mpc8220  mpc8xx  pxa
arm925t  ixp  mips  mpc824x  nios  s3c44b0
[root@gioserver cpu]# cp -a pxa pxa270
[root@gioserver cpu]# ls
74xx_7xx  arm926ejs  lh7a40x  mpc5xx  mpc8260  nios2  s3c44b0
arm720t  at91rm9200  mcf52x2  mpc5xxx  mpc85xx  ppc4xx  sa1100
arm920t  i386  microblaze  mpc8220  mpc8xx  pxa
arm925t  ixp  mips  mpc824x  nios  pxa270
[root@gioserver cpu]#

```

3. board/skku/ 디렉토리 추가

Makefile을 수정한 다음에는 SKKU 보드에 맞는 설정들을 추가한다. 보드 타입에 따른 파일들은 /board 디렉토리에 각 보드 이름의 디렉토리를 만들어서 저장하면 된다. u-boot 1.1.2 버전은 Mainstone 보드를 지원하지 않으므로 PXA250을 사용하는 가장 비슷한 보드인 Lubbock 보드의

디렉토리를 복사해서 사용한다.

```
gurugio@gioserver:/home/SKKU/bulverde/bootloader/u-boot-1.1.2/board
[root@gioserver board]# cp -a lubbock/ skku
[root@gioserver board]# ls skku/ lubbock/
lubbock/:
config.mk flash.c lubbock.c Makefile memsetup.S u-boot.lds

skku/:
config.mk flash.c lubbock.c Makefile memsetup.S u-boot.lds
[root@gioserver board]#
```

그리고 /board/skku/에 있는 lubbock.c 파일을 skku.c 로 바꾸고 Makefile도 lubbock.o를 skku.o 로 바꾼다.

```
gurugio@gioserver:/home/SKKU/bulverde/bootloader/u-boot-1.1.2/board/skku
rm -f tools/gdb/astest tools/gdb/gdbcont tools/gdb/gdbsend
rm -f tools/env/fw_printenv tools/env/fw_setenv
rm -f board/cray/L1/bootscript.c board/cray/L1/bootscript.image
rm -f board/trab/trab_fkt
[root@gioserver u-boot-1.1.2]# cd board/skku/
[root@gioserver skku]# ls
config.mk flash.c Makefile memsetup.S skku.c u-boot.lds
[root@gioserver skku]#
```

```
gurugio@gioserver:/home/SKKU/bulverde/bootloader/u-boot-1.1.2/board/skku
19 # You should have received a copy of the GNU General Public License
20 # along with this program; if not, write to the Free Software
21 # Foundation, Inc., 59 Temple Place, Suite 330, Boston,
22 # MA 02111-1307 USA
23 #
24
25 include $(TOPDIR)/config.mk
26
27 LIB = lib$(BOARD).a
28
29 OBJ := skku.o flash.o
30 SOBJ := memsetup.o
31
32 $(LIB): $(OBJ) $(SOBJ)
33 $(AR) crv $@ $(OBJ) $(SOBJ)
34
35 clean:
```

skku.c를 열어보면 다음과 같이 보드 타입 번호를 지정하는 부분이 있다. LUBBOCK로 된 부분을 SKKU로 수정한다.

```
gurugio@gioserver:/home/SKKU/bulverde/bootloader/u-boot-1.1.2/board/skku
42 /* so we do nothing here */
43
44 /* arch number of SKKU-Board */
45 gd->bd->bi_arch_number = MACH_TYPE_SKKU;
46
47 /* adress of boot parameters */
48 gd->bd->bi_boot_params = 0xa0000100;
"skku.c" 75L, 1984C written
```

4. board/skku/u-boot.lds 수정
cpu/pxa/start.o를 참조하도록 된 부분을 cpu/pxa270/start.o로 고친다.

```
gurugio@gioserver:/home/SKKU/bulverde/bootloader/u-boot-1.1.2
23
24 OUTPUT_FORMAT("elf32-littlearm", "elf32-littlearm", "elf32-littlearm")
25 OUTPUT_ARCH(arm)
26 ENTRY(_start)
27 SECTIONS
28 {
29     . = 0x00000000;
30
31     . = ALIGN(4);
32     .text :
33     {
34         cpu/pxa270/start.o      (.text)
35         *(.text)
36     }
37
38     . = ALIGN(4);
39     .rodata : { *(.rodata) }
"board/skk_u/u-boot.lds" 55L, 1356C written
28,1 57%
```

5. include/asm-arm/mach-type.h 수정
include/asm-arm/mach-types.h 파일을 열어보면 각 보드들에 대한 타입 번호와 보드 타입을 확인하기 위한 매크로들이 들어있다. 여기에 SKKU 보드를 추가한다. 보드 번호는 HYBUS 보드 번호를 사용했다. 이 번호는 리눅스 커널에 넘기기 위한 것이다. 보드 번호 밑에 보드 번호를 확인하기 위한 매크로들도 추가한다.

```
gurugio@gioserver:/home/SKKU/bulverde/bootloader/u-boot-1.1.2/include/asm-arm
625 #define MACH_TYPE_KB9200 612
626 #define MACH_TYPE_SX1 613
627 #define MACH_TYPE_SKKU 65280
628
629
630 #ifdef CONFIG_ARCH_SKKU
631 # ifdef machine_arch_type
632 #  undef machine_arch_type
633 #  define machine_arch_type __machine_arch_type
634 # else
635 #  define machine_arch_type MACH_TYPE_SKKU
636 # endif
637 # define machine_is_skku() (machine_arch_type == MACH_TYPE_SKKU)
638 #else
639 # define machine_is_skku() (0)
640 #endif
641
642
643 #ifdef CONFIG_ARCH_EBSA110
644 # ifdef machine_arch_type
645 #  undef machine_arch_type
"mach-types.h" 7973L, 215306C written
627,14 7%
```

6. include/configs/skk_u.h 생성
include/configs/lubbock.h 를 skku.h로 복사한다. 이 파일이 u-boot를 위한 설정이 들어있는 파일이다. 일단 SKKU 보드를 위한 파일들을 모두 추가하고 그 다음에 이 설정을 보드에 맞춰서 하나씩 맞춰나간다. 지금은 일단 컴파일 테스트를 위해 다음 그림과 같이 보드 이름만 수정하고 #define CONFIG_LCD 1 설정을 #undef CONFIG_LCD로 고친다.

```
gurugio@gioserver:/home/SKKU/bulverde/bootloader/u-boot-1.1.2
42 */
43 #define CONFIG_PXA270      1  /* This is an PXA250 CPU */
44 #define CONFIG_SKKUK       1  /* on an SKKU board */
45 #undef CONFIG_LCD
46 #ifdef CONFIG_LCD
47 #define CONFIG_SHARP_LMSV31
48 #endif
"include/configs/skku.h" 241L, 7192C 45,17 17%
```

7. include/asm-arm/arch-pxa270 생성
include/asm-arm/arch-pxa를 복사해서 include/asm-arm/arch-pxa/arch-pxa270 디렉토리를 만든다.

```
gurugio@gioserver:/home/SKKU/bulverde/bootloader/u-boot-1.1.2/include/asm-arm
arch-arm925t arch-sa1100 hardware.h proc-armv types.h
arch-arm926ejs atomic.h io.h processor.h u-boot-arm.h
arch-at91rm9200 bitops.h mach-types.h ptrace.h u-boot.h
arch-ixp byteorder.h mach-types.h.bak setup.h
[root@gioserver asm-arm]# cp -a arch-pxa arch-pxa270
[root@gioserver asm-arm]# ls
arch-arm720t arch-pxa byteorder.h mach-types.h.bak setup.h
arch-arm920t arch-pxa270 errno.h memory.h sizes.h
arch-arm925t arch-s3c44b0 global_data.h posix_types.h string.h
arch-arm926ejs arch-sa1100 hardware.h proc-armv types.h
arch-at91rm9200 atomic.h io.h processor.h u-boot-arm.h
arch-ixp bitops.h mach-types.h ptrace.h u-boot.h
[root@gioserver asm-arm]#
```

8. 컴파일 옵션 수정
cpu/pxa270/config.mk 를 열어보면 PXA250을 위한 컴파일 설정이 들어있다. 이것을 arm-linux-gcc 3.3.3 버전에 맞도록 다음과 같이 수정한다.

```
gurugio@gioserver:/home/SKKU/bulverde/bootloader/u-boot-1.1.2
12 # the License, or (at your option) any later version.
13 #
14 # This program is distributed in the hope that it will be useful,
15 # but WITHOUT ANY WARRANTY; without even the implied warranty of
16 # MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
17 # GNU General Public License for more details.
18 #
19 # You should have received a copy of the GNU General Public License
20 # along with this program; if not, write to the Free Software
21 # Foundation, Inc., 59 Temple Place, Suite 330, Boston,
22 # MA 02111-1307 USA
23
24
25 PLATFORM_RELFLAGS += -fno-strict-aliasing -fno-common -ffixed-r8 \
26 -msoft-float
27
28 PLATFORM_CPPFLAGS += -mapcs-32 -march=armv5 -mtune=xscale
23,1 Bot
```

9. 이미지 생성
make skku_config 명령으로 SKKU 보드 관련 설정을 실행시키고 make 로 컴파일을 시작한다.

```
gurugio@gioserver:/home/SKKU/bulverde/bootloader/u-boot-1.1.2
[root@gioserver u-boot-1.1.2]# make skku_config
Configuring for skku board...
[root@gioserver u-boot-1.1.2]# make
```

아무 이상이 없이 `u-boot.bin` 파일이 생성되어야 한다. 만약 문제가 발생하면 꼼꼼하게 지금까지의 과정을 다시 확인해보고 컴파일 옵션이나 디렉토리 경로들을 주의해서 확인해본다.

보드 하드웨어에 맞게 부트 로더 설정 하기

Lubbock 보드를 위한 설정 파일들을 복사해서 사용하므로 현재 부트로더 프로그램은 Lubbock 보드의 사양에 맞게 설정되어 있다. 메모리 맵 설정과 메모리 컨트롤러 설정, 네트워크 디바이스 등을 주의해서 설정 파일들을 수정해야 한다. 또 프로세서 초기화 코드들이 PXA250에 맞게 작성되어 있다. PXA270 프로세서도 코어는 PXA250 코어를 사용하므로 크게 달라진 부분은 없지만 프로세서 속도가 빨라졌고 메모리 클럭도 빨라졌으므로 이에 따라 클럭 관련 코드를 수정해 주어야 한다.

가장 먼저 다음 사항들에 대해 기본적으로 이해해야 한다. 하드웨어 회로에서 결정된 사항이므로 이 사항들을 고려해서 부트로더를 설정해야 한다.

- SDRAM의 시작 주소와 크기
 - 시작 주소 : 0xA400 0000, nSDCS1으로 파티션을 선택하므로 0xA000 0000 이 아니라 그 다음 파티션인 0xA400 0000이 된다.
 - 크기 : 0x0400 0000 (64MByte = 32MB X 2개)
- SDRMA의 속도 설정
 - PXA270은 최대 104MHz의 메모리 클럭을 지원한다. SDRAM에 입력되는 클럭을 104MHz로 설정하기 위해서 프로세서는 520MHz의 터보 모드가 되어야 하고 시스템 버스는 208MHz가 되어야 한다. 또 메모리 컨트롤러에 입력되는 클럭이 208MHz가 되어야 SDRAM에 입력되는 클럭이 104MHz가 된다.
- 플래시 메모리의 시작 주소와 크기
 - 시작 주소 : 0x0000 0000, nCS0을 사용해서 첫번째 파티션이 된다.
 - 크기 : 0x0200 0000 (32MByte = 16MB X 2개)
- 플래시 메모리의 속도 설정
 - 플래시 메모리는 비동기로 동작한다. 따라서 속도를 설정할 필요가 없다. PXA270은 동기 플래시 메모리를 지원하므로 플래시 메모리 설정에서 비동기 방식을 사용하기 위한 설정을 명확하게 해주어야 한다.
- 부트 로더가 플래시에 써지는 시작 주소와 크기
 - 0x0000 0000, JTAG 케이블을 통해 플래시에 써주면 0x0번지부터 쓰기 시작한다.
- 부트 로더가 램으로 복사되는 시작 주소와 크기
 - 0xA408 0000 ~ 0xA40995F4, BSS ~0xA40C DF0C
 - 이 값은 부트 로더를 실행시키면 자동으로 계산해서 출력된다. ARM 프로세서에서 리눅스 커널은 (DRAM이 시작하는 물리 주소 + 0x8000)에 로드된다. 또 커널이 부팅 되기 위한 페이지 테이블을 (DRAM이 시작하는 물리 주소 + 0x4000)에 기록하므로 물리 주소의 시작 부분에 부트로더를 로드하지 않도록 한다.
- 부팅 파라미터가 리눅스 커널로 넘겨지는 주소
 - 0xa400 0100, DRAM의 시작 물리 주소 + 0x100
- 커널이 SDRAM에서 실행되는 시작 주소
 - 0xA400 8000, DRAM이 시작하는 물리 주소 + 0x8000이 된다.
- 28F128K3 플래시의 인식
 - u-boot에서는 플래시의 동작을 위한 코드를 사용자가 직접 작성해주어야 한다. 다른 보드들의 플래시 코드를 참조해서 만들면 간단하다. 플래시 메모리의 인식을 위한 값들은 include/flash.h에 저장되어 있다. 플래시를 동작 코드는 board/skku/flash.c에 작성한다.

프로세서의 속도에 대한 설정은 Intel PXA27x Processor Family Developer' s Manual에서 Table 3-7

을 본다. Core Turbo Freq를 520MHz로 맞추기 위한 설정들이 나와있다. 이 테이블에 따라서 레지스터들을 설정해주면 된다.

include/configs/skku.h

가장 먼저 include/configs/skku.h 를 수정한다. 기본적으로 Lubbock 보드에 맞게 되어있지만 이것을 한줄씩 SKKU 보드에 맞게 수정해야 하고 PXA250용 레지스터 설정들을 PXA270에 맞게 고쳐야 한다. 몇가지 새로 추가해야 하는 설정들은 뒤에서 소스들을 살펴 보면서 이야기한다. 다음은 skku.h의 내용이다. 이 파일을 이해하기 위해서는 부트로더 소스에 있는 README 파일을 읽어보는 것이 좋다. 혹시 빼놓거나 잘못 적은 옵션을 찾기 위해서도 README 파일을 자주 읽어보는 것이 필요하다

```
#ifndef __CONFIG_H
#define __CONFIG_H
```

cpu/pxa270/start.S 파일을 열어보면 CONFIG_INIT_CRITICAL가 선언되어 있을 때 cpu_init_crit 함수를 호출하는 부분이 있다. 이 함수는 프로세서의 속도나 메모리 컨트롤러등을 설정하는 일을 하므로 반드시 호출해주어야 한다.

```
/*
 * If we are developing, we might want to start armboot from ram
 * so we MUST NOT initialize critical regs like mem-timing ...
 */
#define CONFIG_INIT_CRITICAL          /* undef for developing */
```

커널에 넘겨주는 태그를 설정한다. ARM 계열 리눅스 커널에서는 램의 물리 주소 + 0x100 번지에 부트로더가 커널에 넘겨주는 데이터를 저장한다. 이 데이터의 구조는 include/asm-arm/setup.h 파일에 struct tag로 정의되어 있다. lib_arm/armlinux.c 파일에서 이 구조체에 값을 쓰는데 소스를 열어보면 if defined (CONFIG_SETUP_MEMORY_TAGS) 등등의 사용자가 정해준 태그 옵션에 따라 데이터를 설정하는 것을 알 수 있다. 보통 없어도 되고 CONFIG_INITRC_TAG만 해주어도 된다.

```
#define CONFIG_INITRD_TAG
```

특별히 하는 일은 없다. 프로세서 종류와 보드 이름을 선언한다.

```
/*
 * High Level Configuration Options
 * (easy to change)
 */
#define CONFIG_PXA270          1          /* This is an PXA270 CPU      */
#define CONFIG_SKKU            1          /* on an SKKU Board         */
#define BOARD_LATE_INIT        1
```

IRQ 설정 코드를 실행시키지 않는다. 부트 로더에서는 굳이 IRQ가 필요없다.

```
#undef CONFIG_USE_IRQ          /* we don't need IRQ/FIQ stuff */
```

전역 변수를 저장하기 위해서 미리 약간의 메모리 영역을 지정해놓는다. start.S 에서 SDRAM을 위한 메모리 맵을 설정하는 부분이 있다. 부트 로더 자신을 SDRAM으로 복사하고 그 외에 환경 설정이나 전역 변수등에 대한 영역, 스택 메모리등의 영역을 설정하는데 사용된다.

```

/*
 * Size of malloc() pool
 */
#define CFG_MALLOC_LEN (CFG_ENV_SIZE + 128*1024)
include/asm-arm/global_data.h에는 gd_t라는 구조체가 있다. 이 구조체는 부트 로더 초기 설정을
기록하기 위해 사용된다. 이 구조체가 저장되는 영역의 크기를 결정한다.
#define CFG_GBL_DATA_SIZE 128 /* size in bytes reserved for initial data */

```

SKKU 보드는 네트워크 장치로 CS8900를 사용한다. 칩 선택 핀에 nCS1을 사용하므로 0x0400 0000 번지로 접근할 수 있다. CS8900칩의 내부 I/O에 접근하기 위해서는 CS8900의 베이스 주소에서 0x300 번지를 더한 주소를 지정해야 한다. 즉 CS8900칩을 I/O 모드로 사용하기 위해서는 0x04000300번지로 사용해야 한다. drivers/ 를 보면 다양한 이더넷 칩의 드라이버가 있다.

```

/*
 * Hardware drivers
 */
#define CONFIG_DRIVER_CS8900 1
#define CS8900_BASE 0x04000300
#define CS8900_BUS16 1

```

시리얼 통신 설정

```

/*
 * select serial console configuration
 */
#define CONFIG_FFUART 1 /* we use FFUART on LUBBOCK */
#define CONFIG_BAUDRATE 115200

```

```

/* allow to overwrite serial and ethaddr */
#define CONFIG_ENV_OVERWRITE

```

u-boot에서 지원하는 명령어 설정. README 파일을 읽어보면 가능한 명령어들에 대한 목록이 있다. NAND, DHCP등 다양한 명령들이 있다. CONFIG_CMD_DFL은 디폴트 명령들을 설정한다.

```

#define CONFIG_COMMANDS (CONFIG_CMD_DFL | CFG_CMD_FAT)

```

```

/* this must be included AFTER the definition of CONFIG_COMMANDS (if any) */
#include <cmd_confdefs.h>

```

```

#define CONFIG_BOOTDELAY 3

```

네트워크 설정

```

#define CONFIG_ETHADDR 12:34:56:78:9a:bc
#define CONFIG_NETMASK 255.255.255.0
#define CONFIG_IPADDR 10.51.15.132
#define CONFIG_SERVERIP 10.51.15.126

```

대기 시간이 지나면 실행하는 명령. 커널 주소를 지정해주면 자동으로 커널로 점프한다.

주석으로 남기면 자동으로 부팅하지 않고 부트 로더의 프롬프트를 출력한다. 지금은 tftp를 통해 커널의 이미지를 다운받아서 0xA4200000 번지에 저장하고 0xA4200000번지로 점프해서 커널을 실행하도록 설정했다.

```
#define CONFIG_BOOTCOMMAND "tftp a4200000 ulmage;bootm a4200000"
```

부팅 옵션. 리눅스의 부팅 옵션과 동일하게 설정된다.

```
#define CONFIG_BOOTARGS "root=/dev/mtdblock2 rootfstype=cramfs\nconsole=ttyS0,115200"
```

```
#define CONFIG_CMDLINE_TAG
```

```
#if (CONFIG_COMMANDS & CFG_CMD_KGDB)
```

```
#define CONFIG_KGDB_BAUDRATE 230400 /* speed to run kgdb serial port */
```

```
#define CONFIG_KGDB_SER_INDEX 2 /* which serial port to use */
```

```
#endif
```

```
/*
```

```
 * Miscellaneous configurable options
```

```
*/
```

```
#define CFG_HUSH_PARSER 1
```

```
#define CFG_PROMPT_HUSH_PS2 "> "
```

```
#define CFG_LONGHELP /* undef to save memory */
```

```
#ifndef CFG_HUSH_PARSER
```

프롬프트 텍스트 설정

```
#define CFG_PROMPT "SKKU2005> " /* Monitor Command Prompt */
```

```
#else
```

```
#define CFG_PROMPT "=> " /* Monitor Command Prompt */
```

```
#endif
```

```
#define CFG_CBSIZE 256 /* Console I/O Buffer Size */
```

```
#define CFG_PBSIZE (CFG_CBSIZE+sizeof(CFG_PROMPT)+16) /* Print Buffer Size */
```

```
#define CFG_MAXARGS 16 /* max number of command args */
```

```
#define CFG_BARGSIZE CFG_CBSIZE /* Boot Argument Buffer Size */
```

```
#define CFG_DEVICE_NULLDEV 1
```

메모리 테스트를 실행할 SDRAM의 시작 주소와 끝 주소를 설정한다. 부트로더의 프롬프트에서 memtest를 치면 실행한다. 주로 커널이 로드될 영역을 테스트해주면 좋다. 만약 커널이 로드될 영역을 테스트했을 때 문제가 발생한다면 커널이 제대로 실행되지 않을 가능성이 높다.

```
#define CFG_MEMTEST_START 0xa4008000 /* memtest works on */
```

```
#define CFG_MEMTEST_END 0xa4080000 /* 0 ~ 8 MB in DRAM */
```

```
#undef CFG_CLKS_IN_HZ /* everything, incl board info, in Hz */
```

커널이 로드될 SDRAM의 주소값. ARM계열 프로세서에서는 반드시 SDRAM의 시작 물리 주소 + 0x8000으로 해야 한다.

```
#define CFG_LOAD_ADDR 0xa4008000 /* default load address */
```

cpu/pxa/interrupt.c에는 udelay_masked라는 함수가 있다. 이 함수는 us 단위로 대기하는 일을 한다. 이 함수에 CFG_HZ 값이 사용되는데 이 값을 OSCR 값과 비교해서 시간이 측정한다. OSCR은 Intel PXA27x Processor Family Developer's Manual의 22.5.5 OSCR0의 설명에서 보듯이 3.25MHz 클럭에 따라 값이 증가한다. 결국 1초에 3250000 증가하고 1us에는 3이 증가하므로 이 값과 비교하면 us 단위로 딜레이를 구현할 수 있다.

```
/* interrupt.c -> get_tbclk(). number of time ticks per second */
#define CFG_HZ 3250000 /* incrementer freq: 3.25MHz */
```

```
/* valid baudrates */
#define CFG_BAUDRATE_TABLE { 9600, 19200, 38400, 57600, 115200 }
```

```
/*
 * Stack sizes
 *
 * The stack sizes are set up in start.S using the settings below
 */
#define CONFIG_STACKSIZE (128*1024) /* regular stack */
#ifdef CONFIG_USE_IRQ
#define CONFIG_STACKSIZE_IRQ(4*1024) /* IRQ stack */
#define CONFIG_STACKSIZE_FIQ(4*1024) /* FIQ stack */
#endif
```

SKKU 보드에서는 nSDCS1을 사용해서 SDRAM의 파티션을 선택하므로 2번째 뱅크가 사용된다. 나머지 뱅크들은 크기를 0x0으로 설정해서 해당 뱅크에 램이 없다는 것을 알려준다.

```
/*
 * Physical Memory Map
 */
#define CONFIG_NR_DRAM_BANKS 1 /* we have 1 banks of DRAM */
#define PHYS_SDRAM_1 0xa0000000 /* SDRAM Bank #1 */
#define PHYS_SDRAM_1_SIZE 0x00000000 /* No SDRAM at 1st partition */
#define PHYS_SDRAM_2 0xa4000000 /* SDRAM Bank #2 */
#define PHYS_SDRAM_2_SIZE 0x04000000 /* 64 MB */
#define PHYS_SDRAM_3 0xa8000000 /* SDRAM Bank #3 */
#define PHYS_SDRAM_3_SIZE 0x00000000 /* 0 MB */
#define PHYS_SDRAM_4 0xac000000 /* SDRAM Bank #4 */
#define PHYS_SDRAM_4_SIZE 0x00000000 /* 0 MB */
```

28F128J3A와 28F128K3는 128KB의 섹터 크기를 가지며 128개의 블록으로 이루어져있다. ARM 계열의 프로세서들은 리셋후 메모리의 0번지를 참조하므로 반드시 0번지에 롬을 붙여야 한다. 플래시를 두개 사용하므로 섹터 크기가 256KB가 된다.

```
/*
 * SKKU uses two '28F128K3' , 128Mbit = 16MB
 * 128Blocks X 1Mbit(128KB)
 */
#define PHYS_FLASH_1 0x00000000 /* Flash Bank #1 */
```

```

#define PHYS_FLASH_2          0x04000000 /* Flash Bank #2 */
#define PHYS_FLASH_SIZE      0x02000000 /* 32 MB */
#define PHYS_FLASH_BANK_SIZE 0x02000000 /* 32 MB Banks */
#define PHYS_FLASH_SECT_SIZE 0x00040000 /* 256 KB sectors (x2) */

/*
 * FLASH and environment organization
 */
#define CFG_MAX_FLASH_BANKS    1          /* max number of memory banks
 */
#define CFG_MAX_FLASH_SECT 128 /* max number of sectors on one chip */

```

플래시와 SDRAM의 시작 주소와 크기 설정

```

#define CFG_DRAM_BASE          PHYS_SDRAM_2
#define CFG_DRAM_SIZE          PHYS_SDRAM_2_SIZE

#define CFG_FLASH_BASE         PHYS_FLASH_1
#define CFG_FLASH_SIZE         PHYS_FLASH_BANK_SIZE

```

부트 로더의 환경 변수를 저장할 메모리 영역은 커널이나 부트 로더와 겹치지 않도록 잘 계산해야 한다.

```

/* FIXME */
#define CFG_ENV_IS_IN_FLASH    1
#define CFG_ENV_ADDR           (PHYS_FLASH_1 + 0x300000) /* Addr of Environment
Sector */
#define CFG_ENV_SIZE           0x40000 /* Total Size of Environment Sector */

```

```

#define FPGA_REGS_BASE_PHYSICAL 0x08000000

```

여기부터는 프로세서와 메모리 컨트롤러를 초기화하기 위한 레지스터들의 값들이다. 매우 중요한 값으로 한 비트가 잘못되어도 보드가 동작하지 않거나 최소한 오동작을 할 위험이 높아지므로 신중하게 계산해서 값을 찾아야 한다. 다른 유사 보드들의 값을 많이 참조해서 최대한 정확한 값을 찾도록 해야 한다.

GPIO 사용 설정. 부트로더에서는 FFUART와 최소한의 장치만 사용한다.

```

/*
 * GPIO settings
 */
#define CFG_GPSR0_VAL          0x00008004
#define CFG_GPSR1_VAL          0x00020080
#define CFG_GPSR2_VAL          0x0001FFFF
#define CFG_GPCR0_VAL          0x00000000
#define CFG_GPCR1_VAL          0x00000380
#define CFG_GPCR2_VAL          0x00000000
#define CFG_GPDR0_VAL          0x0060A800

```

```
#define CFG_GPDR1_VAL          0x00000380
#define CFG_GPDR2_VAL          0x0001C000
#define CFG_GAFR0_L_VAL        0x98400000
#define CFG_GAFR0_U_VAL        0x00002950
#define CFG_GAFR1_L_VAL        0x0000a950
#define CFG_GAFR1_U_VAL        0x0005AAAA
#define CFG_GAFR2_L_VAL        0xA0000000
#define CFG_GAFR2_U_VAL        0x00000002
```

PSSR[RDH] = 0, PSSR[PH] = 0 GPIO를 활성화시킨다. PSSR[BFS] = 1 배터리 폴트가 발생하면 프로세서를 슬립 모드로 바꾼다.

```
#define CFG_PSSR_VAL          0x20
```

nCS<1:0>으로 연결된 플래시 메모리에 대한 설정

```
/*
 * Memory settings
 */
#define CFG_MSC0_VAL          0x7ff973ff3 /* for Flash */
nCS<5:2>에는 아무것도 연결되지 않음
#define CFG_MSC1_VAL          0x00000000
#define CFG_MSC2_VAL          0x00000000
```

프로세서의 동작 클럭을 520MHz(터보 모드)로 설정하기 위한 CCCR 레지스터 설정 값

```
/* for CCCR register */
#define CFG_CPUSPEED          0x0a000290          /* set core clock to 520 MHz */
```

14번 보조프로세서의 6번 레지스터인 CLKCFG 레지스터를 설정한다. CLKCFG 레지스터는 프로세서를 터보 모드로 전환하고 시스템 버스의 클럭을 설정하는데 사용한다.

```
/* ADDED BY GIO for start.S */
#define CFG_CLKCFG_VAL          0x0000000b /* B=1, HT=0, F=1, T=1 in CP14*/
```

SDRAM 컨트롤러를 설정한다. 보드에서 사용하는 SDRAM의 매뉴얼을 주의깊게 읽고 필요한 설정 값들을 찾아야 한다.

```
/*
 * SDRAM partition 1 enable,
 * 2 banks X 13 ROWs X 9 Column X 32 bit width X 2 chips
 * 4 internal banks per 1 chips
 * Partition 1 enable
 * tRP=3, CL=3, tRCD=3, tRAS=7, tRC=11
 * Normal addressing mode
 * MA<24:10> are used
 */
#define CFG_MDCNFG_VAL          0x08000bca /* for SDRAM */
```

CLK_MEM이 208MHz이므로 이 클럭을 반으로 나누어서 SDRAM에 공급해준다.

```
/*
```

```

* SRAM uses SDCLK<1>
* Flash uses SDCLK<0>
* According to CCCR, CLK_MEM is 208MHz,
* thus it should be divided by 2 to be used for SDRAM(104Mhz) -> K1DB2=1
*/
#define CFG_MDREFR_VAL          0x0103801e

/* These value is dummy value.
* There value does not affect SDRAM setting */
#define CFG_MDMRS_VAL           0x00000000

비동기 방식으로 플래시를 사용하므로 모두 0으로 설정한다.
/* Asynchronous flash memory for SKKU */
#define CFG_SXCNFG_VAL 0x00000000

/*
* PCMCIA and CF Interfaces
*/
#define CFG_MECR_VAL            0x00000000 /* No PC Card */
#define CFG_MCMEM0_VAL          0x00010504
#define CFG_MCMEM1_VAL          0x00010504
#define CFG_MCATT0_VAL          0x00010504
#define CFG_MCATT1_VAL          0x00010504
#define CFG_MCIO0_VAL           0x00004715
#define CFG_MCIO1_VAL           0x00004715

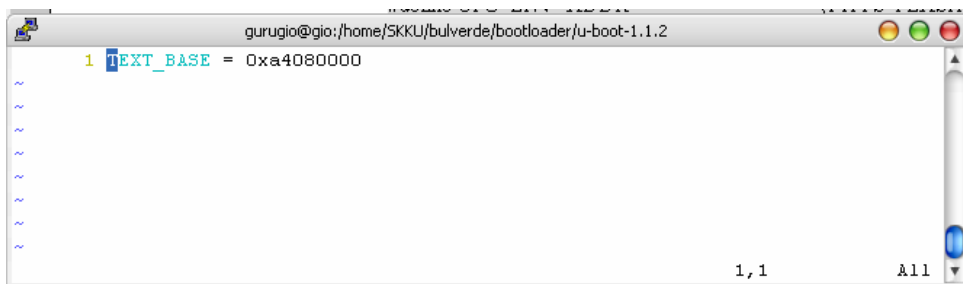
/* timeout values are in ticks */
#define CFG_FLASH_ERASE_TOUT     (25*CFG_HZ) /* Timeout for Flash Erase */
#define CFG_FLASH_WRITE_TOUT    (25*CFG_HZ) /* Timeout for Flash Write */

/* clock */
#define CFG_CKEN_VAL             0x01FFFFFF

#endif /* __CONFIG_H */

```

board/skku/config.mk의 내용은 다음과 같다. 부트 로더가 SDRAM에 로드될 주소를 정해주면 된다. 0xA4008000에 커널이 로드되므로 커널 영역과 멀리 떨어진 곳에 부트 로더를 로드해야 한다.



```
gurugio@gio:/home/SKKU/bulverde/bootloader/u-boot-1.1.2
1 TEXT_BASE = 0xa4080000
~
~
~
~
~
~
~
~
1, 1 Alt
```


프로세서와 메모리 초기화를 위한 레지스터 이해

include/configs/skku.h에는 프로세서와 메모리를 초기화하기 위한 설정 레지스터들의 값들을 적어 주어야 한다. 이러한 설정 레지스터들에 대해 간단하게 살펴본다. 상세하고 완벽한 내용은 Intel PXA27x Processor Family Developer's Manual에서 모두 다루지만 처음부터 영문 매뉴얼을 읽고 이해하는 사람은 많지 않다. 따라서 이 챕터를 읽고 전체적인 내용에 대해서 이해를 하고 그 다음에 영문 매뉴얼을 보면서 자신의 보드에 맞는 값을 찾는 것이 개발하기에 간편하다.

1. 클럭 설정 레지스터

- CCCR : 프로세서의 코어 클럭 설정
 - L : 런 모드 클럭과 오실레이터의 비율
 - 2N : 터보 모드의 클럭과 런 모드 클럭의 비율
 - A : 메모리 컨트롤러 클럭 설정
- CKEN : 프로세서 내부 모듈의 클럭 활성화
 - <22> : 메모리 컨트롤러 클럭
 - <16> : LCD 컨트롤러 클럭
 - <11> : USB 클라이언트, 48MHz 클럭
 - <6> : FFUART 클럭
- CLKCFG : 클럭 모드 변경, Coprocessor 14에 들어있음.
 - T : 터보 모드 활성화
 - F : 클럭 주기 변경 시작
 - HT : 하프 터보 모드
 - B : 시스템 버스 클럭을 런 모드 클럭 주기와 같게 한다. 520MHz 터보 모드에서 런 모드 클럭은 208MHz이다.

2. 메모리 컨트롤러 레지스터

SDRAM을 컨트롤하는 레지스터

- MDCNFG : SDRAM의 파티션에 대한 설정을 한다. 프로세서에 연결된 SDRAM의 인터페이스 속도를 프로세서에 알려주고 프로세서가 SDRAM에 접근할 수 있도록 해주는 레지스터이다. 0/1 파티션과 2/3 파티션으로 나뉘어서 각각 설정한다.
 - DE0 : SDRAM 파티션 0 활성화
 - DE1 : SDRAM 파티션 1 활성화
 - DWID0 : 0/1 파티션에서 데이터 버스 폭
 - DCAC0 : 0/1 파티션에서 주소 비트의 열 개수
 - DRAC0 : 0/1 파티션에서 주소 비트의 행 개수
 - DNB0 : 0/1 파티션에서 뱅크의 개수
 - DTC0 : SDRAM의 AC 타이밍을 설정. 삼성에서 제공하는 칩 매뉴얼을 보면 OPERATING AC PARAMETER라는 테이블이 있고 이 테이블에 DTC0에서 설정하는 tRP, tRCD, tRAS, tRC의 값들이 ns 단위로 나와있다. 104MHz 클럭을 SDRAM에 입력한다면 1클럭은 약 10ns이 될 것이므로 각 설정 값들에 몇 클럭이 필요한지를 알 수 있다. 예를 들어 K4S561632-TC60 칩을 사용한다면 매뉴얼을 참조해서 tRP의 값이 최소 18ns라는 것을 알 수 있다. 따라서 tRP에 2클럭이나 3클럭이 필요하다는 것을 계산 할 수 있다. 삼성에서 출시하는 대부분의 32MB 크기의 SDRAM은 0b11이나 0b10 둘 다 사용될 수 있다.

- DADDR0 : 주소 지정 모드
- DSA1110_0 : SA-1110 주소 지정 방식 호환 모드
- STACK0 : STACK1 비트와 함께 메모리에 사용될 주소 핀을 설정한다.
- MDMRS : MDCNFG 레지스터를 설정한 직후에 MDMRS 레지스터를 설정한다. 보통 디폴트 값 그대로 사용하므로 0x0을 써주기만 한다.
- MDREFR : SDRAM의 리프레시 시간과 동작 클럭을 설정한다. SDCLK, SDCKE를 설정한다.
 - DRI : SDRAM은 한번에 한 행씩 리프레시를 해준다. 이 리프레시의 시간 간격을 지정한다. 삼성에서 출시하는 16/32MB의 SDRAM은 매뉴얼에 따르면 리프레시 시간이 64ms가 되어 한다고 한다. 예를 들어 CLK_MEM가 208MHz일 때, 매뉴얼에 나온 공식대로 $(64ms / 13rows * 208MHz - 31) / 32 = (64 / 13 * 208 - 31) / 32 = 31$ 로 설정하면 된다. 보통 0x1E나 0x1F 값으로 많이 사용한다.
 - K0RUN : SDCLK<0>/<3>의 활성화. SDCLK<0>은 주로 동기 플래시 롬에 사용된다. 만약 비동기 플래시 롬만을 사용한다면 필요가 없다.
 - K0DB2 : SDCLK<0>/<3>의 속도 설정
 - E1PIN : SDCKE 활성화
 - K1RUN : SDCLK<1> 활성화
 - K1DB2 : SDCLK<1> 속도 설정
 - K2RUN : SDCLK<2> 활성화
 - K2DB2 : SDCLK<2> 속도 설정
 - APD : Auto power down 활성화. 램이 없거나 사용하지 않을 때 램 관련 동작들을 멈춰서 전력 소모를 줄인다. 보통 사용하지 않는다.
 - SLFRSH : SDRAM의 Self-refresh를 활성화한다. 메모리 컨트롤러가 여러 장치들을 제어할 때 리프레시 시간을 지키지 못하는 경우가 생길 수 있다. 이럴 때를 대비하여 SDRAM 자체적으로 리프레시를 하도록 하는데 데이터가 안정적인 반면 전력 소모가 늘어난다.
 - K0FREE : SDCLK<0>/<3>의 동작 제어. 1로 설정하면 동작함.

플래시 메모리 관련 설정

- SXCNFG : 동기 플래시 메모리를 사용하는 경우에 설정한다. 비동기 플래시에는 필요없다.
 - SXEN0 : 0번 파티션을 동기 플래시 메모리로 설정. 0으로 설정하면 비동기 플래시를 쓰거나 플래시를 연결하지 않는다는 의미가 된다.
 - SXEN1 : 1번 파티션을 동기 플래시 메모리로 설정
 - SXCL0 : CAS 지연 클럭 수. 각 플래시 메모리의 매뉴얼에 나와있다. 읽기 명령이 플래시에 전달된 시점에서 몇 클럭 후에 데이터가 출력되는지를 지정한다. SDCLK 클럭의 배수로 지정되고 만약 플래시 메모리가 메모리 클럭의 이분지 클럭으로 동작한다면 2배수로 지정해야 한다.
 - SXTPO : 메모리 타입. Burst-of-8이나 Burst-of-16을 선택한다.
 - SXCLEXT0 : 3비트의 SXCL0 값을 4비트값으로 확장하는데 사용한다.
- MSCx : 비동기 플래시 메모리 설정. nCS<5:0>으로 플래시 파티션을 선택하는데 MSCx 레지스터 한 개가 두개의 nCS를 설정한다. 즉 MSC0이 nCS<1:0>을 설정하게 된다. MSC0을 중심으로 설명한다.
 - RT0 : nCS0의 디바이스 타입.
 - 000 Non-burst ROM 이나 플래시 메모리
 - 001 SRAM
 - 010 Burst-of-four ROM 이나 Burst-of-four 플래시 메모리
 - 011 Burst-of-eight ROM 이나 Burst-of-eight 플래시 메모리

- RBW0 : nCS0의 버스 폭, 0이면 32비트 1이면 16비트로 지정된다. 회로도에서 보듯이 32개의 버스 라인이 있으므로 32비트 폭을 가진다.
- RDF0 : nWE 신호를 얼마나 오랫동안 인가해주어야 데이터를 쓸 수 있는지를 말한다. 28F128K3 칩의 매뉴얼에는 WE# write pulse width low 값이 최소 60ns라고 나와있다. 메모리 클럭이 208MHz이므로 12 싸이클이 된다. 최소값이 12싸이클이므로 13싸이클로 정하는게 안전하다. 그런데 Intel PXA27x Processor Family Developer' s Manual에서는 필요한 싸이클에 1을 뺀 값이라고 했으므로 12가 되고 RDF 값이 12이면 15를 쓰라고 나와있으므로 0xF를 써주면 된다.
- RDN0 : 계산 방식은 RDF와 같다. 버스트 모드를 사용할 때만 다르다.
- RRR0 : 버스트 모드가 아닌 경우에 nWE 신호가 인가된 후 다시 nWE 신호가 인가될 수 있는 간격을 말한다. 즉 메모리를 한번 쓰기 위해 nWE가 인가되고 해지된 다음 다시 메모리를 쓰기 위해 nWE 가 인가되는 경우에 nWE가 하이 상태로 있어야 하는 최소 시간 간격을 말한다. 28F128K3 칩의 매뉴얼에는 WE# write pulse width high에 최소 35ns이라고 나와있고 메모리 클럭이 208MHz이므로 7 싸이클이 된다. 이 값에 1을 빼고 반으로 나눈 값인 3을 레지스터에 써주면 된다.
- RBUFF0 : 비동기 플래시를 사용할 때는 0으로 설정한다.

결국 우리 보드에서 작성된 설정 값들은 다음과 같다.

- MSC0_VAL : 0x7ff973ff3
- CPUSPEED : 0x0a000290
- CLKCFG_VAL : 0x0000000b
- MDCNFG_VAL : 0x08000bca
- MDREFR_VAL : 0x0103801e
- MDMRS_VAL : 0x00000000

3. GPIO 설정 레지스터

다음은 Intel PXA27x Processor Family Developer' s Manual에서 24장 General Purpose I/O Controller에 있는 테이블24-2를 가져온 것이다. SKKU 보드에서 사용하는 GPIO와 사용되는 기능을 노락색으로 색칠을 했다. GPIO 관련 설정을 하기 위해서는 이렇게 프로세서에 있는 GPIO들을 놓고 어떤 핀이 어떤 기능으로 사용될지를 먼저 결정하고 그에 맞춰서 레지스터들의 설정값들을 계산하면 된다.

Table 24-2. GPIO Alternate Functions (Sheet 1 of 4)

GPIO Pin	Pin Name	Alternate Function 1 (In)	Alternate Function 2 (In)	Alternate Function 3 (In)	Alternate Function 1 (Out)	Alternate Function 2 (Out)	Alternate Function 3 (Out)
0	GPIO<0>						
1	GPIO<1>/ nRESET_GPIO4						
2	SYS_EN ⁵						
3	GPIO<3>/ PWR_SCL						
4	GPIO<4>/ PWR_SDA						
5	PWR_CAP<0> ⁵						
6	PWR_CAP<1> ⁵						
7	PWR_CAP<2> ⁵						
8	PWR_CAP<3> ⁵						
9	GPIO<9>			FFCTS	HZ_CLK		CHOUT<0>
10	GPIO<10>	FFDCD		USB_P3_5 ⁷	HZ_CLK		CHOUT<1>
11	GPIO<11>	EXT_SYNC<0>	SSPRXD2	USB_P3_1	CHOUT<0>	PWM_OUT<2>	48_MHz
12	GPIO<12>	EXT_SYNC<1>	CIF_DD<7>		CHOUT<1>	PWM_OUT<3>	48_MHz
13	GPIO<13>	CLK_EXT	KP_DKIN<7>	KP_MKIN<7>	SSPTXD2		
14	GPIO<14>	L_VSYNC	SSPSFRM2			SSPSFRM2	UCLK
15	GPIO<15>				nPCE<1>	nCS<1>	
16	GPIO<16>	KP_MKIN<5>				PWM_OUT<0>	FFTXD
17	GPIO<17>	KP_MKIN<6>	CIF_DD<6>			PWM_OUT<1>	
18	GPIO<18>	RDY					
19	GPIO<19>	SSPSCLK2		FFRXD	SSPSCLK2	L_CS	nURST
20	GPIO<20>	DREQ<0>	MBREQ		nSDCS<2>		
21	GPIO<21>				nSDCS<3>	DVAL<0>	MBGNT
22	GPIO<22>	SSPEXTCLK2	SSPSCLK2EN	SSPSCLK2	KP_MKOUT<7>	SSPSYSCLK2	SSPSCLK2
23	GPIO<23>		SSPSCLK		CIF_MCLK	SSPSCLK	
24	GPIO<24>	CIF_FV	SSPSFRM		CIF_FV	SSPSFRM	
25	GPIO<25>	CIF_LV			CIF_LV	SSPTXD	
26	GPIO<26>	SSPRXD	CIF_PCLK	FFCTS			
27	GPIO<27>	SSPEXTCLK	SSPSCLKEN	CIF_DD<0>	SSPSYSCLK		FFRTS
28	GPIO<28>	AC97_BITCLK	I2S_BITCLK	SSPSFRM	I2S_BITCLK		SSPSFRM
29	GPIO<29>	AC97_SDATA_IN_0	I2S_SDATA_IN	SSPSCLK	SSPRXD2		SSPSCLK
30	GPIO<30>				I2S_SDATA_OUT	AC97_SDATA_OUT	USB_P3_2
31	GPIO<31>				I2S_SYNC	AC97_SYNC	USB_P3_6
32	GPIO<32>				MSSCLK	MMCLK	

Table 24-2. GPIO Alternate Functions (Sheet 2 of 4)

GPIO Pin	Pin Name	Alternate Function 1 (In)	Alternate Function 2 (In)	Alternate Function 3 (In)	Alternate Function 1 (Out)	Alternate Function 2 (Out)	Alternate Function 3 (Out)
33	GPIO<33>	FFRXD	FFDSR		DVAL<1>	nCS<5>	MBGNT
34	GPIO<34>	FFRXD	KP_MKIN<3>	SSPSCLK3	USB_P2_2		SSPSCLK3
35	GPIO<35>	FFCTS	USB_P2_1	SSPSFRM3		KP_MKOUT<6>	SSPTXD3
36	GPIO<36>	FFDCD	SSPSCLK2	KP_MKIN<7>	USB_P2_4	SSPSCLK2	
37	GPIO<37>	FFDSR	SSPSFRM2	KP_MKIN<3>	USB_P2_8	SSPSFRM2	FFTXD
38	GPIO<38>	FFRI	KP_MKIN<4>	USB_P2_3	SSPTXD3	SSPTXD2	PWM_OUT<1>
39	GPIO<39>	KP_MKIN<4>		SSPSFRM3	USB_P2_6	FFTXD	SSPSFRM3
40	GPIO<40>	SSPRXD2		USB_P2_5	KP_MKOUT<6>	FFDTR	SSPSCLK3
41	GPIO<41>	FFRXD	USB_P2_7	SSPRXD3	KP_MKOUT<7>	FFRTS	
42	GPIO<42>	BTRXD	ICP_RXD				CIF_MCLK
43	GPIO<43>			CIF_FV	ICP_TXD	BTTXD	CIF_FV
44	GPIO<44>	BTCTS		CIF_LV			CIF_LV
45	GPIO<45>			CIF_PCLK	AC97_SYSCLK	BTRTS	SSPSYSCLK3
46	GPIO<46>	ICP_RXD	STD_RXD			PWM_OUT<2>	
47	GPIO<47>	CIF_DD<0>			STD_TXD	ICP_TXD	PWM_OUT<3>
48	GPIO<48>	CIF_DD<5>			BB_OB_DAT<1>	nPOE	
49	GPIO<49>					nPWE	
50	GPIO<50>	CIF_DD<3>		SSPSCLK2	BB_OB_DAT<2>	nPIOR	SSPSCLK2
51	GPIO<51>	CIF_DD<2>			BB_OB_DAT<3>	nPIOW	
52	GPIO<52>	CIF_DD<4>	SSPSCLK3		BB_OB_CLK	SSPSCLK3	
53	GPIO<53>	FFRXD	USB_P2_3		BB_OB_STB	CIF_MCLK	SSPSYSCLK
54	GPIO<54>		BB_OB_WAIT	CIF_PCLK		nPCE<2>	
55	GPIO<55>	CIF_DD<1>	BB_IB_DAT<1>			nPREG	
56	GPIO<56>	nPWAIT	BB_IB_DAT<2>		USB_P3_4		
57	GPIO<57>	nIOIS16	BB_IB_DAT<3>				SSPTXD
58	GPIO<58>		LDD<0>			LDD<0>	
59	GPIO<59>		LDD<1>			LDD<1>	
60	GPIO<60>		LDD<2>			LDD<2>	
61	GPIO<61>		LDD<3>			LDD<3>	
62	GPIO<62>		LDD<4>			LDD<4>	
63	GPIO<63>		LDD<5>			LDD<5>	
64	GPIO<64>		LDD<6>			LDD<6>	
65	GPIO<65>		LDD<7>			LDD<7>	
66	GPIO<66>		LDD<8>			LDD<8>	
67	GPIO<67>		LDD<9>			LDD<9>	
68	GPIO<68>		LDD<10>			LDD<10>	

Table 24-2. GPIO Alternate Functions (Sheet 3 of 4)

GPIO Pin	Pin Name	Alternate Function 1 (In)	Alternate Function 2 (In)	Alternate Function 3 (In)	Alternate Function 1 (Out)	Alternate Function 2 (Out)	Alternate Function 3 (Out)
69	GPIO<69>		LDD<11>			LDD<11>	
70	GPIO<70>		LDD<12>			LDD<12>	
71	GPIO<71>		LDD<13>			LDD<13>	
72	GPIO<72>		LDD<14>			LDD<14>	
73	GPIO<73>		LDD<15>			LDD<15>	
74	GPIO<74>					L_FCLK_RD	
75	GPIO<75>					L_LCLK_A0	
76	GPIO<76>					L_PCLK_WR	
77	GPIO<77>					L_BIAS	
78	GPIO<78>				nPCE<2>	nCS<2>	
79	GPIO<79>				PSKTSEL	nCS<3>	PWM_OUT<2>
80	GPIO<80>	DREQ<1>	MBREQ			nCS<4>	PWM_OUT<3>
81	GPIO<81>		CIF_DD<0>		SSPTXD3	BB_OB_DAT<0>	
82	GPIO<82>	SSPRXD3	BB_IB_DAT<0>	CIF_DD<5>			FFDTR
83	GPIO<83>	SSPSFRM3	BB_IB_CLK	CIF_DD<4>	SSPSFRM3	FFTXD	FFRTS
84	GPIO<84>	SSPCLK3	BB_IB_STB	CIF_FV	SSPCLK3		CIF_FV
85	GPIO<85>	FFRXD	DREQ<2>	CIF_LV	nPCE<1>	BB_IB_WAIT	CIF_LV
86	GPIO<86>	SSPRXD2	LDD<16>	USB_P3_5	nPCE<1>	LDD<16>	
87	GPIO<87>	nPCE<2>	LDD<17>	USB_P3_1	SSPTXD2	LDD<17>	SSPSFRM2
88	GPIO<88>	USBHPWR<1>	SSPRXD2	SSPSFRM2			SSPSFRM2
89	GPIO<89>	SSPRXD3		FFRI	AC97_SYSCLK	USBHPEN<1>	SSPTXD2
90	GPIO<90>	KP_MKIN<5>	USB_P3_5	CIF_DD<4>		nURST	
91	GPIO<91>	KP_MKIN<6>	USB_P3_1	CIF_DD<5>		UCLK	
92	GPIO<92>	MMDAT<0>			MMDAT<0>	MSBS	
93	GPIO<93>	KP_DKIN<0>	CIF_DD<6>		AC97_SDATA_OUT		
94	GPIO<94>	KP_DKIN<1>	CIF_DD<5>		AC97_SYNC		
95	GPIO<95>	KP_DKIN<2>	CIF_DD<4>	KP_MKIN<6>	AC97_RESET_n		
96	GPIO<96>	KP_DKIN<3>	MBREQ	FFRXD		DVAL<1>	KP_MKOUT<6>
97	GPIO<97>	KP_DKIN<4>	DREQ<1>	KP_MKIN<3>		MBGNT	
98	GPIO<98>	KP_DKIN<5>	CIF_DD<0>	KP_MKIN<4>	AC97_SYSCLK		FFRTS
99	GPIO<99>	KP_DKIN<6>	AC97_SDATA_IN_1	KP_MKIN<5>			FFTXD
100	GPIO<100>	KP_MKIN<0>	DREQ<2>	FFCTS			
101	GPIO<101>	KP_MKIN<1>					
102	GPIO<102>	KP_MKIN<2>		FFRXD	nPCE<1>		
103	GPIO<103>	CIF_DD<3>				KP_MKOUT<0>	

Table 24-2. GPIO Alternate Functions (Sheet 4 of 4)

GPIO Pin	Pin Name	Alternate Function 1 (In)	Alternate Function 2 (In)	Alternate Function 3 (In)	Alternate Function 1 (Out)	Alternate Function 2 (Out)	Alternate Function 3 (Out)
104	GPIO<104>	CIF_DD<2>			PSKTSEL	KP_MKOUT<1>	
105	GPIO<105>	CIF_DD<1>			nPCE<2>	KP_MKOUT<2>	
106	GPIO<106>	CIF_DD<9>				KP_MKOUT<3>	
107	GPIO<107>	CIF_DD<8>				KP_MKOUT<4>	
108	GPIO<108>	CIF_DD<7>			CHOUT<0>	KP_MKOUT<5>	
109	GPIO<109>	MMDAT<1>	MSSDIO		MMDAT<1>	MSSDIO	
110	GPIO<110>	MMDAT<2>/ MMCCS<0>			MMDAT<2>/ MMCCS<0>		
111	GPIO<111>	MMDAT<3>/ MMCCS<1>			MMDAT<3>/ MMCCS<1>		
112	GPIO<112>	MMCMD	nMSINS		MMCMD		
113	GPIO<113>			USB_P3_3	I2S_SYSCCLK	AC97_RESET_n	
114 ¹	GPIO<114>	CIF_DD<1>			UEN	UVS0	
115 ²	GPIO<115>	DREQ<0>	CIF_DD<3>	MBREQ	UEN	nUVS1	PWM_OUT<1>
116 ³	GPIO<116>	CIF_DD<2>	AC97_SDATA_IN_0	UDET	DVAL<0>	nUVS2	MBGNT
117	GPIO<117>	SCL			SCL		
118	GPIO<118>	SDA			SDA		
119	GPIO<119>	USBHPWR<2>					
120	GPIO<120>					USBHPEN<2>	

다음은 GPIO 관련 레지스터들의 설명이다.

- GPDR : 입출력 방향을 결정한다.
 - GPDR0<0>은 GPIO<1>을 설정한다. 마찬가지로 GPDR1<0>은 GPIO<32>를 설정한다. 즉 한 비트에 한 핀을 설정한다.
 - SKKU 보드에서 설정한 값은 다음과 같다.
 - GPDR<0> : 0xC0F393E4
 - GPDR<1> : 0xFCEFA8B3
 - GPDR<2> : 0xE2F1FFFF
 - GPDR<3> : 0xFE1FFFE5
- GPSR : GPIO 핀 셋팅 레지스터
 - GPIO 핀이 출력으로 설정되어 있고 GPSR에서 해당 핀이 1로 설정되면 하이 신호를 내보낸다.
 - SKKU 보드에서 설정한 값은 다음과 같다.
 - GPSR<0> : 0x00008004
 - GPSR<1> : 0x00020080
 - GPSR<2> : 0x16C14000
 - GPSR<3> : 0x0003E000
- GPCR : GPIO 핀 클리어 레지스터
 - GPIO 핀이 출력으로 설정되어 있고 GPSR에서 해당 핀을 1로 설정하면 로우 신호를 내보낸다.
 - SKKU 보드에서 설정한 값은 다음과 같다.
 - GPCR<0> : 0x0
 - GPCR<1> : 0x00000380, FFUART에서 로우 신호가 필요하다.
 - GPCR<2> : 0x0
 - GPCR<3> : 0x0

※GPCR과 GPSR에 값을 쓸 때는 가장 마지막으로 설정한 값을 따른다. 즉 GPCR<0>과 GPSR<0>을 둘 다 1로 설정해줘도 GPCR<0>을 나중에 써줬으면 로우 신호가 나가고 GPSR<0>을 나중에 써주면 하이 신호가 출력된다.

- GRER : 상승 엣지 확인 활성화
 - 해당 핀에서 상승 엣지가 검출되면 GEDR 레지스터의 해당 비트를 1로 설정한다.
 - SKKU 보드에서 설정한 값은 다음과 같다.
 - GRER<0> : 0x0
 - GRER<1> : 0x0
 - GRER<2> : 0x00080000, MMC_CD에서 필요하다.
 - GRER<3> : 0x0
- GFER : 하강 엣지 확인 활성화
 - 해당 핀에서 하강 엣지가 검출되면 GEDR 레지스터의 해당 비트를 1로 설정한다.
 - SKKU 보드에서 설정한 값은 다음과 같다.
 - GFER<0> : 0x00000001, On/Off 스위치에 사용됨.
 - GFER<1> : 0x0
 - GFER<2> : 0x04000000, PCMCIA의 인터럽트에 사용
 - GFER<3> : 0x00000008, USB의 인터럽트에 사용
- GAFR : GPIO 핀의 기능 설정
 - GPIO 핀이 일반 입출력이 아니라 프로세서 내부의 유닛에 의해 사용되도록 설정한다. 보통 세가지의 기능을 선택할 수 있고 일반 GPIO로 사용할 수 있으므로 총 4가지 경우가 있다. 따라서 한 핀당 두 개의 비트로 설정한다.
 - 예를 들어, GPIO<0>은 GAFR0_L<1:0>으로 설정할 수 있고 GPIO<16>은 GAFR0_U<1:0>으로 설정한다. 또 GPIO<32>는 GAFR1_L<1:0>으로 설정한다.
 - GAFR0_L : 0x830C0000
 - GAFR0_U : 0xA520051A
 - GAFR1_L : 0x999A955A
 - GAFR1_U : 0xAAA5A0AA
 - GAFR2_L : 0x6A8AAAAA
 - GAFR2_U : 0x0109A002
 - GAFR3_L : 0x5400100A
 - GAFR3_U : 0x00001409
- GPLR : 핀 상태 확인
 - 현재 핀의 신호 레벨을 알 수 있다. 읽기 전용 레지스터이다.
- GEDR : 엣지 검출 상태
 - GRER이나 GFER의 설정에 따라 엣지가 검출되면 1로 셋팅된다.

프로세서와 메모리 초기화

1. 프로세서 속도 설정

SDRAM의 동작 클럭 설정

다음 테이블에 필요한 설정이 나와있다.

Core Run Freq (MHz)	CLKCFG[T]	Core Turbo Freq (MHz)	CLKCFG[T]	CLKCFG[HT]	CCCR[L]	CCCR[2N]	System Bus (MHz)	CLKCFG[B]	CLK_MEM (MHz)	CCCR[A]	SDCLK<2:1> SDRAM Clocks (MHz)	MDREFR[KxDB2]	Synchronous Flash (MHz)	MDREFR[K0DB4]	MDREFR[K0DB2]	LCD(MHz)
208	0	510	1	0	16	5	208	1	208	1	104	1	52	1	X	104
208	0	510	1	0	16	5	104	0	104	1	52	1	52	1	X	104

삼성의 K4S561632D-TL75 SDRAM은 최대 133MHz의 클럭으로 동작할 수 있다. PXA27x에서 지원하는 SDRAM 클럭은 104MHz와 52MHz이므로 두 클럭 중에 원하는 속도로 사용하면 된다.

이 테이블에서 터보 모드로 진입하기 위해서는 CLKCFG[T] = 1, CLKCFG[HT] = 0, CCCR[L] = 16, CCCR[2N] = 5로 설정해주면 된다. 그 다음은 프로세서의 모드 이외에 관련 클럭들의 설정이다. 예를 들어, CLKCFG[B] = 1이면 시스템 버스가 208MHz가 되고 0으로 설정하면 104MHz가 된다. 마찬가지로 CCCR[A]가 1이면 메모리 클럭 설정에 관련된 CLK_MEM가 208MHz가 되는 식이다. SDRAM을 104MHz로 동작시키기 위해서는 CCCR[A]와 MDREFR[KxDB2]가 1이 되어야 한다. 현재 SKKU 보드에서는 CCCR[A] = 0, MDREFR[KxDB2] = 0으로 설정해서, CLK_MEM을 104MHz로 동작시키고 SDRAM을 52MHz로 동작시킨다.

일반적으로 PXA270은 520MHz로 동작하는 터보 모드를 사용한다. 이런 터보 모드를 사용하기 위해 필요한 레지스터 설정을 주의해서 봐야 한다. 다음 장에 설명하는 시피유와 메모리 컨트롤러 초기화 코드를 세밀하게 살펴보고 해당 레지스터의 설정에 주의해야 한다.

PXA270에 맞게 초기화 코드 수정하기

u-boot 1.1.2 를 포팅하기 위해서는 다음 파일들을 수정해야 한다. 그 외에 코드들은 큰 흐름만 파악하고 있으면 별다른 문제없이 보드에 포팅할 수 있다.

- board/skku/memsetup.S : 메모리 컨트롤러 설정. SDRAM의 속도와 리프레시 주기, 플래시 메모리 동작에 대한 설정을 한다.
- board/skku/skku.c : 특정 보드에 맞는 하드웨어 설정을 한다. 커널에 넘기기 위한 부팅 파라미터의 주소를 설정하고 SDRAM의 시작 주소와 크기를 설정한다. ARM용 리눅스 커널에서는 주로 SDRAM의 시작 주소 + 0x100 번지에 파라미터를 넘긴다.
- board/skku/flash.c : 보드에서 사용하는 플래시 메모리의 동작에 관련된 함수들. u-boot에서는 flash_init, flash_erase, flash_print_info, write_buff을 공통적으로 작성해야 한다. 이 함수들은 보드마다 공통적으로 존재해야 하고 그 내부 코드는 보드에 사용된 플래시에 따라 다르게 작성해주면 된다. common/flash.c와 common/cmd_flash.c 파일에서 이 소스에 작성된 함수들을 사용한다.
- cpu/pxa270/start.S : 프로세서의 속도 설정과 SDRAM으로 부트 로더 자신을 복사하는 일을 한다.

실행되는 순서는 start.S 에서 memsetup.S 파일을 호출한다. 그리고 메모리 컨트롤러 설정이 끝나면 부트 로더 자신을 SDRAM으로 복사하고 SDRAM으로 점프한다. 그러면 lib_arm/board.c 파일에 들어있는 start_armboot 함수가 실행되고 include/configs/skku.h에서 설정한 내용에 따라 나머지 하드웨어들에 대한 설정을 하고 사용자의 명령을 기다린다. 사용자가 커널을 로드하도록 명령하면 lib_arm/armlinux.c에 있는 do_bootm_linux 함수가 호출되고 커널 이미지를 로드하고 커널로 점프한다.

다음은 SKKU 보드를 위해 Lubbock 보드의 코드를 수정해서 만든 소스들이다. 소스들을 보면서 부트로더의 동작을 이해하도록 한다. u-boot는 코드의 문서화가 매우 잘 되어있다. 주석을 이해하면 대부분의 동작을 이해할 수 있다. 따라서 주로 하드웨어에 따라 달라지고 포팅을 위해 수정할 필요가 있는 부분들을 중심으로 한글로 설명을 적는다.

cpu/pxa270/start.S

```
/*
 * armboot – Startup Code for XScale
 *
 * Copyright (C) 1998  Dan Malek <dmalek@jlc.net>
 * Copyright (C) 1999  Magnus Damm <kieraypc01.p.y.kie.era.ericsson.se>
 * Copyright (C) 2000  Wolfgang Denk <wd@denx.de>
 * Copyright (C) 2001  Alex Zuepke <azu@sysgo.de>
 * Copyright (C) 2002  Kyle Harris <kharris@nexus-tech.net>
 * Copyright (C) 2003  Robert Schwebel <r.schwebel@pengutronix.de>
 * Copyright (C) 2003  Kai-Uwe Bloem <kai-uwe.bloem@auerswald.de>
 *
 * See file CREDITS for list of people who contributed to this
 * project.
 *
 * This program is free software; you can redistribute it and/or
 * modify it under the terms of the GNU General Public License as
 * published by the Free Software Foundation; either version 2 of
 * the License, or (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program; if not, write to the Free Software
 * Foundation, Inc., 59 Temple Place, Suite 330, Boston,
 * MA 02111-1307 USA
 */
```

```
#include <config.h>
#include <version.h>
```

```
.globl _start
_start: b reset
        ldr    pc, _undefined_instruction
        ldr    pc, _software_interrupt
        ldr    pc, _prefetch_abort
        ldr    pc, _data_abort
        ldr    pc, _not_used
        ldr    pc, _irq
        ldr    pc, _fiq
```

```

_undefined_instruction: .word undefined_instruction
_software_interrupt:    .word software_interrupt
_prefetch_abort: .word prefetch_abort
_data_abort:            .word data_abort
_not_used:              .word not_used
_irq:                   .word irq
_fiq:                   .word fiq

```

```

        .balignl 16,0xdeadbeef

```

```

/*
 * Startup Code (reset vector)
 *
 * do important init only if we don't start from RAM!
 * - relocate armboot to ram
 * - setup stack
 * - jump to second stage
 */

```

```

_TEXT_BASE:
        .word   TEXT_BASE

```

```

.globl _armboot_start
_armboot_start:
        .word _start

```

```

/*
 * These are defined in the board-specific linker script.
 */
.globl _bss_start
_bss_start:
        .word __bss_start

```

```

.globl _bss_end
_bss_end:
        .word _end

```

```

#ifdef CONFIG_USE_IRQ
/* IRQ stack memory (calculated at run-time) */
.globl IRQ_STACK_START
IRQ_STACK_START:
        .word   0x0badc0de

```

```

/* IRQ stack memory (calculated at run-time) */
.globl FIQ_STACK_START

```

```

FIQ_STACK_START:
    .word 0x0badc0de
#endif

```

```

/*****
/*
/* the actual reset code
/*
*****/

```

부트 로더가 시작되는 지점
reset:

```

    mrs    r0,cpsr          /* set the cpu to SVC32 mode      */
    bic    r0,r0,#0x1f      /* (supervisor mode, M=10011)  */
    orr    r0,r0,#0x13
    msr    cpsr,r0

```

```

/*
 * we do sys-critical inits only at reboot,
 * not when booting from ram!
 */

```

```
#ifdef CONFIG_INIT_CRITICAL
```

하드웨어를 초기화하는 대부분의 동작을 cpu_init_crit 함수에서 실행한다. 따라서 반드시 CONFIG_INIT_CRITICAL을 선언하도록 한다.

```
    bl     cpu_init_crit    /* we do sys-critical inits    */

```

```
#endif
```

```

relocate:                /* relocate U-Boot to RAM      */
    adr    r0, _start      /* r0 ← current position of code */
    ldr    r1, _TEXT_BASE  /* test if we run from flash or RAM */
    cmp    r0, r1          /* don't reloc during debug     */
    beq    stack_setup

```

```

    ldr    r2, _armboot_start
    ldr    r3, _bss_start
    sub    r2, r3, r2       /* r2 ← size of armboot        */
    add    r2, r0, r2       /* r2 ← source end address     */

```

```

copy_loop:
    ldmia  r0!, {r3-r10}    /* copy from source address [r0] */
    stmia  r1!, {r3-r10}    /* copy to target address [r1]   */
    cmp    r0, r2           /* until source end address [r2] */
    ble    copy_loop

```

```

        /* Set up the stack */
stack_setup:
        ldr    r0, _TEXT_BASE        /* upper 128 KiB: relocated uboot */
        sub    r0, r0, #CFG_MALLOC_LEN /* malloc area */
        sub    r0, r0, #CFG_GBL_DATA_SIZE /* bdfinfo */
#ifdef CONFIG_USE_IRQ
        sub    r0, r0, #(CONFIG_STACKSIZE_IRQ+CONFIG_STACKSIZE_FIQ)
#endif
        sub    sp, r0, #12          /* leave 3 words for abort-stack */

clear_bss:
        ldr    r0, _bss_start        /* find start of bss segment */
        ldr    r1, _bss_end          /* stop here */
        mov    r2, #0x00000000       /* clear */

clbss_l:str    r2, [r0]              /* clear loop... */
        add    r0, r0, #4
        cmp    r0, r1
        bne    clbss_l

```

이 코드는 SKKU 보드에서 프로세서와 주변 장치들의 인터페이스를 위해 부착한 버퍼를 설정하는 코드이다. SKKU 보드의 CPLD와 관련이 있는 코드이므로 다른 보드에서는 무시해도 좋다.

```

        /* open buffer chips, 74LVC162444A, 74LVC16245A */
        ldr    r8, =0x04100000
        mov    r9, #1
        str    r9, [r8, #0]

        ldr    pc, _start_armboot

_start_armboot: .word start_armboot

```

```

/*****
/*
/* CPU_init_critical registers
/*
/* - setup important registers
/* - setup memory timing
/*
*****/

```

초기화에 필요한 몇 가지 레지스터들의 주소를 선언한다.

```

/* Interrupt-Controller base address */
IC_BASE: .word 0x40d00000

```

```

#define ICMR    0x04

/* Reset-Controller */
RST_BASE:      .word    0x40f00030
#define RCSR    0x00

/* Operating System Timer */
OSTIMER_BASE:  .word    0x40a00000
#define OSMR3   0x0C
#define OSCR    0x10
#define OWER    0x18
#define OIER    0x1C

/* Clock Manager Registers */
#ifdef CFG_CPUSPEED
CC_BASE:       .word    0x41300000
#define CCCR    0x00
cpuspeed:      .word    CFG_CPUSPEED
#else
#error "You have to define CFG_CPUSPEED!!"
#endif

/* RS: ??? */
.macro CPWAIT
mrc    p15,0,r0,c2,c0,0
mov    r0,r0
sub    pc,pc,#4
.endm

cpu_init_crit:

/* mask all IRQs */
ldr    r0, IC_BASE
mov    r1, #0x00
str    r1, [r0, #ICMR]

#ifdef CFG_CPUSPEED
CCCR 레지스터의 값을 위해 선언한 CFG_CPUSPEED 값을 CCCR 레지스터에 저장한다.
/* set clock speed */
ldr    r0, CC_BASE
ldr    r1, cpuspeed
str    r1, [r0, #CCCR]
/* set CCCR register */

CLKCFG 레지스터에 CFG_CLKCFG_VAL 값을 저장한다.

```

```

ldr r0, =CFG_CLKCFG_VAL          /* for Turbo mode */
mcr    p14, 0, r0, c6, c0, 0     /* enter the frequency change sequence */

```

CLKCFG 레지스터의 F 비트를 1로 설정하면 터보 모드로 전환된다. 따라서 이 지점부터는 터보 모드로 동작한다.

```

/* NOW Turbo-mode is on! */

```

```

setspeed_done:
#endif

```

board/skku/memsetup.S에 있는 memsetup 함수를 실행한다. 이 함수에서 메모리 컨트롤러를 초기화하므로 실행중 동작이 멈추는 경우가 많다. memsetup 함수를 가장 주의깊게 살펴보아야 한다.

```

/*
 * before relocating, we have to setup RAM timing
 * because memory timing is board-dependend, you will
 * find a memsetup.S in your board directory.
 */
mov    ip,    lr
bl     memsetup
mov    lr,    ip

```

리눅스 커널이 부팅하려면 MMU가 비활성 상태이어야 한다. D-cache는 반드시 꺼져있어야 하고 I-cache는 상관하지 않는다.

```

/* Memory interfaces are working. Disable MMU and enable I-cache. */

```

```

ldr    r0, =0x2001                /* enable access to all coproc. */
mcr    p15, 0, r0, c15, c1, 0
CPWAIT

```

```

mcr    p15, 0, r0, c7, c10, 4     /* drain the write & fill buffers */
CPWAIT

```

```

mcr    p15, 0, r0, c7, c7, 0      /* flush Icache, Dcache and BTB */
CPWAIT

```

```

mcr    p15, 0, r0, c8, c7, 0      /* flush instuction and data TLBs */
CPWAIT

```

```

/* Enable the Icache */
mrc    p15, 0, r0, c1, c0, 0
orr     r0, r0, #0x1800
mcr    p15, 0, r0, c1, c0, 0
CPWAIT

```

cpu_init_crit 함수가 끝나고 relocate로 돌아간다.

```

mov    pc, lr

```



```

/*****
/*
/* Interrupt handling
/*
/*
*****/

/* IRQ stack frame
*/

#define S_FRAME_SIZE 72

#define S_OLD_R0      68
#define S_PSR         64
#define S_PC          60
#define S_LR          56
#define S_SP          52

#define S_IP          48
#define S_FP          44
#define S_R10         40
#define S_R9          36
#define S_R8          32
#define S_R7          28
#define S_R6          24
#define S_R5          20
#define S_R4          16
#define S_R3          12
#define S_R2          8
#define S_R1          4
#define S_R0          0

#define MODE_SVC 0x13

/* use bad_save_user_regs for abort/prefetch/undef/swi ... */

.macro bad_save_user_regs
sub    sp, sp, #S_FRAME_SIZE
stmia  sp, {r0 - r12}          /* Calling r0-r12 */
add    r8, sp, #S_PC

ldr    r2, _armboot_start
sub    r2, r2, #(CONFIG_STACKSIZE+CFG_MALLOC_LEN)
sub    r2, r2, #(CFG_GBL_DATA_SIZE+8) @ set base 2 words into abort stack
ldmia  r2, {r2 - r4}          /* get pc, cpsr, old_r0 */
add    r0, sp, #S_FRAME_SIZE  /* restore sp_SVC */

```

```

add    r5, sp, #S_SP
mov     r1, lr
stmia   r5, {r0 - r4}          /* save sp_SVC, lr_SVC, pc, cpsr, old_r */
mov     r0, sp
.endm

```

```

/* use irq_save_user_regs / irq_restore_user_regs for          */
/* IRQ/FIQ handling                                           */

```

```

.macro irq_save_user_regs
sub     sp, sp, #S_FRAME_SIZE
stmia   sp, {r0 - r12}        /* Calling r0-r12      */
add     r8, sp, #S_PC
stmdb   r8, {sp, lr}^         /* Calling SP, LR     */
str     lr, [r8, #0]          /* Save calling PC    */
mrs     r6, spsr
str     r6, [r8, #4]          /* Save CPSR          */
str     r0, [r8, #8]          /* Save OLD_R0        */
mov     r0, sp
.endm

```

```

.macro irq_restore_user_regs
ldmia   sp, {r0 - lr}^        @ Calling r0 - lr
mov     r0, r0
ldr     lr, [sp, #S_PC]        @ Get PC
add     sp, sp, #S_FRAME_SIZE
subs    pc, lr, #4             @ return & move spsr_svc into cpsr
.endm

```

```

.macro get_bad_stack
ldr     r13, _armboot_start    @ setup our mode stack
sub     r13, r13, #(CONFIG_STACKSIZE+CFG_MALLOC_LEN)
sub     r13, r13, #(CFG_GBL_DATA_SIZE+8) @ reserved a couple spots in abort
stack

```

```

str     lr, [r13]              @ save caller lr / spsr
mrs     lr, spsr
str     lr, [r13, #4]

```

```

mov     r13, #MODE_SVC        @ prepare SVC-Mode
msr     spsr_c, r13
mov     lr, pc
movs    pc, lr
.endm

```

```

        .macro get_irq_stack                                @ setup IRQ stack
        ldr    sp, IRQ_STACK_START
        .endm

        .macro get_fiq_stack                                @ setup FIQ stack
        ldr    sp, FIQ_STACK_START
        .endm

/*****
/*
/* exception handlers
/*
/*
*****/

        .align    5
undefined_instruction:
        get_bad_stack
        bad_save_user_regs
        bl        do_undefined_instruction

        .align    5
software_interrupt:
        get_bad_stack
        bad_save_user_regs
        bl        do_software_interrupt

        .align    5
prefetch_abort:
        get_bad_stack
        bad_save_user_regs
        bl        do_prefetch_abort

        .align    5
data_abort:
        get_bad_stack
        bad_save_user_regs
        bl        do_data_abort

        .align    5
not_used:
        get_bad_stack
        bad_save_user_regs
        bl        do_not_used

```

```

#ifdef CONFIG_USE_IRQ

    .align    5
irq:
    get_irq_stack
    irq_save_user_regs
    bl        do_irq
    irq_restore_user_regs

    .align    5
fiq:
    get_fiq_stack
    irq_save_user_regs        /* someone ought to write a more */
    bl        do_fiq          /* efficient fiq_save_user_regs */
    irq_restore_user_regs

#else

    .align    5
irq:
    get_bad_stack
    bad_save_user_regs
    bl        do_irq

    .align    5
fiq:
    get_bad_stack
    bad_save_user_regs
    bl        do_fiq

#endif

/*****
/*
/* Reset function: the PXA250 doesn't have a reset function, so we have to */
/* perform a watchdog timeout for a soft reset. */
/*
*****/

    .align    5
.globl reset_cpu

    /* FIXME: this code is PXA250 specific. How is this handled on */
    /* other XScale processors? */

reset_cpu:

```

```
/* We set OWE:WME (watchdog enable) and wait until timeout happens */
```

```
ldr    r0, OSTIMER_BASE
ldr    r1, [r0, #OWER]
orr    r1, r1, #0x0001          /* bit0: WME          */
str    r1, [r0, #OWER]
```

```
/* OS timer does only wrap every 1165 seconds, so we have to set */
/* the match register as well.                                   */
```

```
ldr    r1, [r0, #OSCR]          /* read OS timer      */
add    r1, r1, #0x800            /* let OSMR3 match after */
add    r1, r1, #0x800            /* 4096*(1/3.6864MHz)=1ms */
str    r1, [r0, #OSMR3]
```

```
reset_endless:
```

```
b      reset_endless
```

board/skku/memsetup.S

```
/*
 * Most of this taken from Redboot hal_platform_setup.h with cleanup
 *
 * NOTE: I haven't clean this up considerably, just enough to get it
 * running. See hal_platform_setup.h for the source. See
 * board/cradle/memsetup.S for another PXA250 setup that is
 * much cleaner.
 *
 * See file CREDITS for list of people who contributed to this
 * project.
 *
 * This program is free software; you can redistribute it and/or
 * modify it under the terms of the GNU General Public License as
 * published by the Free Software Foundation; either version 2 of
 * the License, or (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program; if not, write to the Free Software
 * Foundation, Inc., 59 Temple Place, Suite 330, Boston,
 * MA 02111-1307 USA
 */

#include <config.h>
#include <version.h>
#include <asm/arch/pxa-regs.h>

DRAM_SIZE: .long CFG_DRAM_SIZE

/* wait for coprocessor write complete */
.macro CPWAIT reg
    mrc    p15,0,Wreg,c2,c0,0
    mov    Wreg,Wreg
    sub    pc,pc,#4
.endm

/*
 *      Memory setup
 */
```

```
.globl memsetup
memsetup:
```

```
    mov     r10, lr
```

GPIO 설정을 가장 먼저 한다.GPIO에서 핀 기능을 잘못 설정하면 외부 장치들이 동작하지 않는 경우가 있으므로 자신이 쓰고자 하는 장치들에 대한 설정에 주의해야 한다.

```
    /* Set up GPIO pins first ----- */
```

```
    ldr     r0,     =GPSR0
    ldr     r1,     =CFG_GPSR0_VAL
    str     r1,     [r0]
```

```
    ldr     r0,     =GPSR1
    ldr     r1,     =CFG_GPSR1_VAL
    str     r1,     [r0]
```

```
    ldr     r0,     =GPSR2
    ldr     r1,     =CFG_GPSR2_VAL
    str     r1,     [r0]
```

```
    ldr     r0,     =GPCR0
    ldr     r1,     =CFG_GPCR0_VAL
    str     r1,     [r0]
```

```
    ldr     r0,     =GPCR1
    ldr     r1,     =CFG_GPCR1_VAL
    str     r1,     [r0]
```

```
    ldr     r0,     =GPCR2
    ldr     r1,     =CFG_GPCR2_VAL
    str     r1,     [r0]
```

```
    ldr     r0,     =GPDR0
    ldr     r1,     =CFG_GPDR0_VAL
    str     r1,     [r0]
```

```
    ldr     r0,     =GPDR1
    ldr     r1,     =CFG_GPDR1_VAL
    str     r1,     [r0]
```

```
    ldr     r0,     =GPDR2
    ldr     r1,     =CFG_GPDR2_VAL
    str     r1,     [r0]
```

```

ldr        r0,      =GAFR0_L
ldr        r1,      =CFG_GAFR0_L_VAL
str        r1,      [r0]

ldr        r0,      =GAFR0_U
ldr        r1,      =CFG_GAFR0_U_VAL
str        r1,      [r0]

ldr        r0,      =GAFR1_L
ldr        r1,      =CFG_GAFR1_L_VAL
str        r1,      [r0]

ldr        r0,      =GAFR1_U
ldr        r1,      =CFG_GAFR1_U_VAL
str        r1,      [r0]

ldr        r0,      =GAFR2_L
ldr        r1,      =CFG_GAFR2_L_VAL
str        r1,      [r0]

ldr        r0,      =GAFR2_U
ldr        r1,      =CFG_GAFR2_U_VAL
str        r1,      [r0]

ldr        r0,      =PSSR          /* enable GPIO pins */
ldr        r1,      =CFG_PSSR_VAL
str        r1,      [r0]

/* ----- */
/* Enable memory interface */
/*
/* The sequence below is based on the recommended init steps
/* detailed in the Intel PXA250 Operating Systems Developers Guide,
/* Chapter 10.
/* ----- */

/* ----- */
/* Step 1: Wait for at least 200 microseconds to allow internal
/* clocks to settle. Only necessary after hard reset...
/* FIXME: can be optimized later
/* ----- */

ldr r3, =OSCR                      /* reset the OS Timer Count to zero */
mov r2, #0
str r2, [r3]

```



```

        ldr r4, =0x300                /* really 0x2E1 is about 200usec, */
                                      /* so 0x300 should be plenty */
1:
        ldr r2, [r3]
        cmp r4, r2
        bgt 1b

```

mem_init:

여기서부터가 SDRAM과 플래시 메모리의 동작을 설정하는 코드이다. 가장 많이 수정해야 하는 부분이므로 자세하게 살펴보고 이해해야 한다.

```

        ldr    r1, =MEMC_BASE          /* get memory controller base addr. */

        /* ----- */
        /* Step 2a: Initialize Asynchronous static memory controller */
        /* ----- */

        /* MSC registers: timing, bus width, mem type */

```

플래시 메모리 설정을 위한 MSC 레지스터들을 설정한다.

```

        /* MSC0: nCS(0,1) */
        ldr    r2, =CFG_MSC0_VAL
        str    r2, [r1, #MSC0_OFFSET]
        ldr    r2, [r1, #MSC0_OFFSET] /* read back to ensure */
                                      /* that data latches */

        /* MSC1: nCS(2,3) */
        ldr    r2, =CFG_MSC1_VAL
        str    r2, [r1, #MSC1_OFFSET]
        ldr    r2, [r1, #MSC1_OFFSET]

        /* MSC2: nCS(4,5) */
        ldr    r2, =CFG_MSC2_VAL
        str    r2, [r1, #MSC2_OFFSET]
        ldr    r2, [r1, #MSC2_OFFSET]

```

PCMCIA나 메모리 카드들에 대한 설정을 한다. 부트로더에서 이런 장치들을 사용하지 않는다면 코드를 없애도 상관이 없다.

```

        /* ----- */
        /* Step 2b: Initialize Card Interface */
        /* ----- */

        /* MECR: Memory Expansion Card Register */
        ldr    r2, =CFG_MECR_VAL
        str    r2, [r1, #MECR_OFFSET]
        ldr    r2, [r1, #MECR_OFFSET]

```

```

/* MCMEM0: Card Interface slot 0 timing */
ldr    r2, =CFG_MCMEM0_VAL
str     r2, [r1, #MCMEM0_OFFSET]
ldr     r2, [r1, #MCMEM0_OFFSET]

/* MCMEM1: Card Interface slot 1 timing */
ldr     r2, =CFG_MCMEM1_VAL
str     r2, [r1, #MCMEM1_OFFSET]
ldr     r2, [r1, #MCMEM1_OFFSET]

/* MCATT0: Card Interface Attribute Space Timing, slot 0 */
ldr     r2, =CFG_MCATT0_VAL
str     r2, [r1, #MCATT0_OFFSET]
ldr     r2, [r1, #MCATT0_OFFSET]

/* MCATT1: Card Interface Attribute Space Timing, slot 1 */
ldr     r2, =CFG_MCATT1_VAL
str     r2, [r1, #MCATT1_OFFSET]
ldr     r2, [r1, #MCATT1_OFFSET]

/* MCIO0: Card Interface I/O Space Timing, slot 0 */
ldr     r2, =CFG_MCIO0_VAL
str     r2, [r1, #MCIO0_OFFSET]
ldr     r2, [r1, #MCIO0_OFFSET]

/* MCIO1: Card Interface I/O Space Timing, slot 1 */
ldr     r2, =CFG_MCIO1_VAL
str     r2, [r1, #MCIO1_OFFSET]
ldr     r2, [r1, #MCIO1_OFFSET]

```

DMA동작을 위한 FLYCNFG 레지스터를 설정한다. 보통 부트로더에서는 DMA가 필요없으므로 아무런 코드도 없다.

```

/* ----- */
/* Step 2c: Write FLYCNFG  FIXME: what's that??? */
/* ----- */

/* ----- */
/* Step 2d: Initialize Timing for Sync Memory (SDCLK0) */
/* ----- */

/* Before accessing MDREFR we need a valid DRI field, so we set */
/* this to power on defaults + DRI field. */

```

MDREFR은 여러 번에 걸쳐서 특정 비트 필드씩 나눠서 설정해준다. 여기서는 DRI 필드와 디폴트로 셋팅되어있는 비트들만 설정한다.

```
ldr    r3,    =CFG_MDREFR_VAL
0xFFFF로 마스킹을 하므로 0번 비트부터 11번 비트까지, 즉 DRI 필드만 남기고 나머지는 무시한다.
```

```
ldr    r2,    =0xFFF
and    r3,    r3, r2
ldr    r4,    =0x03ca4000
orr    r4,    r4, r3
str    r4,    [r1, #MDREFR_OFFSET] /* write back MDREFR */
ldr    r4,    [r1, #MDREFR_OFFSET]
```

```
/* Note: preserve the mdrefr value in r4 */
```

```
/* ----- */
/* Step 3: Initialize Synchronous Static Memory (Flash/Peripherals) */
/* ----- */
```

```
/* Initialize SXCNFG register. Assert the enable bits */
```

```
/* Write SXMRS to cause an MRS command to all enabled banks of */
/* synchronous static memory. Note that SXLCR need not be written */
/* at this time. */
```

```
/* FIXME: we use async mode for now */
```

동기 플래시 메모리를 사용할 때 여기서 플래시와 프로세서의 동기를 위한 설정을 한다. SKKU보드에서는 비동기 플래시만을 사용하므로 0x0값을 써준다.

```
ldr    r2,    =CFG_SXCNFG_VAL
str    r2,    [r1, #SXCNFG_OFFSET]
ldr    r2,    [r1, #SXCNFG_OFFSET]
```

```
/* ----- */
/* Step 4: Initialize SDRAM */
/* ----- */
```

이 코드들은 대부분 PXA250고 달라지는 코드이다. 설정의 순서가 바뀌어도 보드가 동작하지 않는 경우가 생기므로 주의해서 수정해나가야 한다. 만약 이 코드들 중간에서 보드가 멈춘다면 멈추기 직전에 설정한 비트들이 하드웨어와 맞지 않는 것이므로 이전에 설정한 값을 다시 살펴본다.

MDREFR의 설정 중에서 KORUN, E0PIN, K0DB2, DRI, APD, SLFRSH 비트들만 설정한다.

```
ldr    r3,    =CFG_MDREFR_VAL
ldr    r0,    =(MDREFR_KORUN | MDREFR_E0PIN | MDREFR_K0DB2)
and    r3,    r3, r0
ldr    r2,    [r1, #MDREFR_OFFSET]
ldr    r0,    =(0xFFF | MDREFR_APD | MDREFR_SLFRSH)
and    r2,    r2, r0
orr    r0,    r3, r2
```

```

str      r3, [r1, #MDREFR_OFFSET]
ldr      r3, [r1, #MDREFR_OFFSET]

```

```

nop
nop
nop

```

K1RUN, K2RUN, K1DB2, K2DB2 비트만 설정한다.

```

ldr      r3, [r1, #MDREFR_OFFSET]
ldr      r0, =CFG_MDREFR_VAL
and      r0, r0, #(MDREFR_K1RUN | MDREFR_K2RUN | MDREFR_K1DB2 |
MDREFR_K2DB2)
orr      r3, r3, r0
str      r3, [r1, #MDREFR_OFFSET]

```

SKKU보드에서는 SLFRSH 기능을 사용하지 않으므로 SLFRSH 비트를 지워주었다.

```

ldr      r3, [r1, #MDREFR_OFFSET]
bic      r3, r3, #MDREFR_SLFRSH
str      r3, [r1, #MDREFR_OFFSET]
ldr      r3, [r1, #MDREFR_OFFSET]

```

E1PIN 비트를 설정한다.

```

ldr      r0, =CFG_MDREFR_VAL
and      r0, r0, #MDREFR_E1PIN
orr      r3, r3, r0
str      r3, [r1, #MDREFR_OFFSET]
ldr      r3, [r1, #MDREFR_OFFSET]

```

```

/* Step 4d: write MDCNFG with MDCNFG:DEx deasserted (set to 0), to
/*          configure but not enable each SDRAM partition pair. */

```

MDCNFG 레지스터를 초기화하지만 만약 DE1 비트가 1이라면 DE1 비트는 지운다. 한꺼번에 두개의 파티션을 모두 초기화할 수 없기 때문이다.

```

ldr      r4, =CFG_MDCNFG_VAL
bic      r4, r4, #(MDCNFG_DE0|MDCNFG_DE1)

str      r4, [r1, #MDCNFG_OFFSET] /* write back MDCNFG */
ldr      r4, [r1, #MDCNFG_OFFSET]

```

```

/* Step 4e: Wait for the clock to the SDRAMs to stabilize,
/*          100..200  $\mu$ sec. */

```

```

ldr r3, =OSCR /* reset the OS Timer Count to zero */
mov r2, #0

```

```

        str r2, [r3]
        ldr r4, =0x300                                /* really 0x2E1 is about 200usec, */
                                                    /* so 0x300 should be plenty */

1:        ldr r2, [r3]
        cmp r4, r2
        bgt 1b

/* Step 4f: Trigger a number (usually 8) refresh cycles by */
/*          attempting non-burst read or write accesses to disabled */
/*          SDRAM, as commonly specified in the power up sequence */
/*          documented in SDRAM data sheets. The address(es) used */
/*          for this purpose must not be cacheable. */

        ldr    r3,    =CFG_DRAM_BASE
        str    r2,    [r3]
        str    r2,    [r3]
        str    r2,    [r3]
        str    r2,    [r3]
        str    r2,    [r3]
        str    r2,    [r3]
        str    r2,    [r3]
        str    r2,    [r3]

/* Step 4g: Write MDCNFG with enable bits asserted */
/*          (MDCNFG:DEx set to 1). */

MDCNFG에서 DE1 비트를 설정한다. 즉 파티션 1에 있는 SDRAM을 설정한다.
        ldr    r3,    [r1, #MDCNFG_OFFSET]
        orr    r3,    r3,    #(MDCNFG_DE0|MDCNFG_DE1)
        str    r3,    [r1, #MDCNFG_OFFSET]

/* Step 4h: Write MDMRS. */

        ldr    r2,    =CFG_MDMRS_VAL
        str    r2,    [r1, #MDMRS_OFFSET]

/* We are finished with Intel's memory controller initialisation */

/* ----- */
/* Disable (mask) all interrupts at interrupt controller */

```

```
/* ----- */
```

initirqs:

```
mov    r1, #0          /* clear int. level register (IRQ, not FIQ) */
ldr    r2, =ICLR
str    r1, [r2]

ldr    r2, =ICMR        /* mask all interrupts at the controller */
str    r1, [r2]
```

```
/* ----- */
/* Clock initialisation */
/* ----- */
```

initclks:

```
/* Disable the peripheral clocks, and set the core clock frequency */
/* (hard-coding at 398.12MHz for now). */

/* Turn Off ALL on-chip peripheral clocks for re-configuration */
/* Note: See label 'ENABLECLKS' for the re-enabling */
ldr    r1, =CKEN
/* enable memory controller, OS TIMER, FFUART, */
ldr    r2, =CFG_CKEN_VAL
str    r2, [r1]

/* ----- */
/* FIXED BY GIO for PXA270 */
/* Every setting for Turbo mode is already done in start.S */
/* ----- */

/* default value in case no valid rotary switch setting is found */
//ldr  r2, =(CCCR_L16|CCCR_LCD|CCCR_N25) /* 520Mhz, Turbo-mode */

/* ... and write the core clock config register */
//ldr  r1, =CCCR
//str  r2, [r1]
```

#ifdef RTC

```
/* enable the 32Khz oscillator for RTC and PowerManager */

ldr    r1, =OSCC
mov    r2, #OSCC_OON
```

```

        str    r2, [r1]

        /* NOTE:  spin here until OSCC.OOK get set, meaning the PLL      */
        /* has settled.                                                    */
60:      ldr     r2, [r1]
        ands   r2, r2, #1
        beq    60b
#endif

        /* ----- */
-- */
        /* ----- */
-- */

        /* Save SDRAM size */
        ldr    r1, =DRAM_SIZE
        str    r8, [r1]

        /* Interrupt init: Mask all interrupts */
        ldr    r0, =ICMR /* enable no sources */
        mov    r1, #0
        str    r1, [r0]

        /* FIXME */

#define NODEBUG
#ifndef NODEBUG
        /*Disable software and data breakpoints */
        mov    r0, #0
        mcr    p15,0,r0,c14,c8,0 /* ibcr0 */
        mcr    p15,0,r0,c14,c9,0 /* ibcr1 */
        mcr    p15,0,r0,c14,c4,0 /* dbcon */

        /*Enable all debug functionality */
        mov    r0, #0x80000000
        mcr    p14,0,r0,c10,c0,0 /* dcsr */

#endif

        /* ----- */
-- */
        /* End memsetup */
        /* ----- */
-- */

```

endmemsetup:

mov pc, lr

board/skku/skku.c

```
/*
 * (C) Copyright 2002
 * Kyle Harris, Nexus Technologies, Inc. kharris@nexus-tech.net
 *
 * (C) Copyright 2002
 * Sysgo Real-Time Solutions, GmbH <www.elinos.com>
 * Marius Groeger <mgroeger@sysgo.de>
 *
 * See file CREDITS for list of people who contributed to this
 * project.
 *
 * This program is free software; you can redistribute it and/or
 * modify it under the terms of the GNU General Public License as
 * published by the Free Software Foundation; either version 2 of
 * the License, or (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program; if not, write to the Free Software
 * Foundation, Inc., 59 Temple Place, Suite 330, Boston,
 * MA 02111-1307 USA
 */
```

```
#include <common.h>
```

```
/* ----- */
```

```
/*
 * Miscelaneous platform dependent initialisations
 */
```

```
int board_init (void)
{
```

```
    DECLARE_GLOBAL_DATA_PTR;
```

```
    /* memory and cpu-speed are setup before relocation */
    /* so we do _nothing_ here */
```

커널에 전달할 데이터들을 설정한다. 보드의 등록 번호와 부팅 파라미터를 저장한 메모리

번호를 전달한다.

```
    /* arch number of SKKU */  
    gd->bd->bi_arch_number = MACH_TYPE_SKKU;  
  
    /* adress of boot parameters */  
    gd->bd->bi_boot_params = 0xa4000100;  
  
    return 0;  
}
```

```
int board_late_init(void)  
{  
    setenv("stdout", "serial");  
    setenv("stderr", "serial");  
    return 0;  
}
```

```
int dram_init (void)  
{  
    DECLARE_GLOBAL_DATA_PTR;
```

보드에서 사용하는 DRAM에 맞게 수정한다. SKKU보드는 한 개의 뱅크만을 사용하므로 나머지 뱅크들에 대한 설정은 지웠다.

```
    /* ADD BY GIO */  
    /* SKKU board has only one bank */  
    gd->bd->bi_dram[0].start = CFG_DRAM_BASE;  
    gd->bd->bi_dram[0].size = CFG_DRAM_SIZE;  
  
    return 0;  
}
```

board/skku/flash.c

```
/*
 * (C) Copyright 2001
 * Kyle Harris, Nexus Technologies, Inc. kharris@nexus-tech.net
 *
 * (C) Copyright 2001
 * Wolfgang Denk, DENX Software Engineering, wd@denx.de.
 *
 * See file CREDITS for list of people who contributed to this
 * project.
 *
 * This program is free software; you can redistribute it and/or
 * modify it under the terms of the GNU General Public License as
 * published by the Free Software Foundation; either version 2 of
 * the License, or (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program; if not, write to the Free Software
 * Foundation, Inc., 59 Temple Place, Suite 330, Boston,
 * MA 02111-1307 USA
 */
```

```
#include <common.h>
#include <linux/byteorder/swab.h>
```

플래시 메모리의 각 뱅크에 관한 정보를 관리한다. SKKU보드는 하나의 뱅크만 사용하므로 include/configs/skku.h에서 CFG_MAX_FLASH_BANKS를 1로 정의해주었다. flash_info 구조체는 include/flash.h 파일에 선언되어 있다.

```
flash_info_t flash_info[CFG_MAX_FLASH_BANKS]; /* info for FLASH chips */
```

```
/* Board support for 1 or 2 flash devices */
#define FLASH_PORT_WIDTH32
#undef FLASH_PORT_WIDTH16

#ifndef FLASH_PORT_WIDTH16
#define FLASH_PORT_WIDTH        ushort
#define FLASH_PORT_WIDTHV      vu_short
#define SWAP(x)                  __swab16(x)
#else
#define FLASH_PORT_WIDTH        ulong
```

```

#define FLASH_PORT_WIDTHV          vu_long
#define SWAP(x)                    __swab32(x)
#endif

#define FPW      FLASH_PORT_WIDTH
#define FPWV     FLASH_PORT_WIDTHV

#define mb() __asm__ __volatile__ (" : : : \"memory\" )

/*-----
--
* Functions
*/
static ulong flash_get_size (FPW *addr, flash_info_t *info);
static int write_data (flash_info_t *info, ulong dest, FPW data);
static void flash_get_offsets (ulong base, flash_info_t *info);
void inline spin_wheel (void);

/*-----
--
*/

unsigned long flash_init (void)
{
    int i;
    ulong size = 0;

    for (i = 0; i < CFG_MAX_FLASH_BANKS; i++) {
        switch (i) {
            case 0:
                첫 번째 뱅크를 초기화한다.
                flash_get_size ((FPW *) PHYS_FLASH_1, &flash_info[i]);
                flash_get_offsets (PHYS_FLASH_1, &flash_info[i]);
                break;
            case 1:
                flash_get_size ((FPW *) PHYS_FLASH_2, &flash_info[i]);
                flash_get_offsets (PHYS_FLASH_2, &flash_info[i]);
                break;
            default:
                플래시 메모리 뱅크가 2개가 넘으면 안된다.
                panic ("configured too many flash banks!Wn");
                break;
        }
        size += flash_info[i].size;
    }
}

```

```

/* Protect monitor and environment sectors
*/
monitor_flash_len 변수는 lib_arm/board.c 파일에서 _bss_start - _armboot_start로 계산하고
있다. _bss_start 와 _armboot_start 값들은 cpu/pxa270/start.S 파일에 정의되어 있는데
부트로더에서 임시로 사용할 지역 변수들을 저장하는 곳이다. 따라서 부트로더의 환경 변수를
저장할 영역과 함께 쓰기 금지를 시켜서 데이터를 보호해야 한다.
    flash_protect ( FLAG_PROTECT_SET,
                    CFG_FLASH_BASE,
                    CFG_FLASH_BASE + monitor_flash_len - 1,
                    &flash_info[0] );

    flash_protect ( FLAG_PROTECT_SET,
                    CFG_ENV_ADDR,
                    CFG_ENV_ADDR + CFG_ENV_SIZE - 1, &flash_info[0] );
부트로더가 실행되면 지금까지 설정한 플래시 메모리의 정보들이 출력된다. 만약 부팅
메시지를 줄이고 싶다면 이 코드를 없애도 되지만 개발 중에는 아주 유용하게 쓰인다.
부트로더가 쓰기 금지로 설정한 섹터에는 커널을 다운로드 할 수 없으므로 부트로더를
포팅하기 전부터 플래시 메모리의 메모리 맵을 미리 구상해두는 것이 좋다.
    flash_print_info(&flash_info[0]);

    return size;
}

/*-----
*/
static void flash_get_offsets (ulong base, flash_info_t *info)
{
    int i;

    if (info->flash_id == FLASH_UNKNOWN) {
        printf("flash_get_offsets() failed.. %s %dWn", __FILE__, __LINE__);
        return;
    }
    플래시 메모리의 각 섹터의 시작 주소를 지정한다. 섹터의 시작 주소는 섹터 크기의 배수가 될
    것이다. 또 섹터마다 쓰지 못하게 막는 플래그가 있는데 이 값을 0으로 초기화한다.
    if ((info->flash_id & FLASH_VENDMASK) == FLASH_MAN_INTEL) {
        for (i = 0; i < info->sector_count; i++) {
            info->start[i] = base + (i * PHYS_FLASH_SECT_SIZE);
            info->protect[i] = 0;
        }
    }
}

/*-----
--
*/

```

```

void flash_print_info (flash_info_t *info)
{
    int i;

    if (info->flash_id == FLASH_UNKNOWN) {
        printf ("missing or unknown FLASH typeWn");
        return;
    }

    switch (info->flash_id & FLASH_VENDMASK) {
    case FLASH_MAN_INTEL:
        printf ("INTEL ");
        break;
    default:
        printf ("Unknown Vendor ");
        break;
    }

    switch (info->flash_id & FLASH_TYPEMASK) {
    case FLASH_28F128J3A:
        printf ("28F128J3AWn");
        break;

    /* FIX BY GIO */
    case FLASH_28F128K3:
        printf("28F128K3Wn");
        break;

    default:
        printf ("Unknown Chip TypeWn");
        break;
    }

    printf ("  Size: %ld MB in %d SectorsWn",
            info->size >> 20, info->sector_count);

    printf ("  Sector Start Addresses:");

```

각 섹터별 시작 주소와 쓰기 금지가 되어있는지 여부를 출력해준다.

```

    for (i = 0; i < info->sector_count; ++i) {
        if ((i % 5) == 0)
            printf ("Wn  ");
        printf (" %08IX%s",
                info->start[i],
                info->protect[i] ? " (RO)" : " ");
    }

```

```

        printf ("Wn");
        return;
    }

    /*
     * The following code cannot be run from FLASH!
     */
    static ulong flash_get_size (FPW *addr, flash_info_t *info)
    {

```

```

        volatile FPW value;

```

플래시 메모리에 명령해서 플래시 메모리의 장치 정보를 읽어온다. 이 명령어 값은 플래시 메모리의 매뉴얼에서 설명된 값이고 하드웨어적으로 고정된 값이므로 신경 쓰지 않아도 좋다.

```

        /* Write auto select command: read Manufacturer ID */
        addr[0x5555] = (FPW) 0x00AA00AA;
        addr[0x2AAA] = (FPW) 0x00550055;
        //addr[0x5555] = (FPW) 0x00900090;
        addr[0] = (FPW) 0x00900090;

```

```

        mb ();
        value = addr[0];

```

```

        switch (value) {

```

SKKU보드는 인텔의 플래시 메모리를 사용한다. 따라서 부트로더를 부팅했을 때 info->flash_id = 300000 이라는 메시지가 출력되어야 한다.

```

        case (FPW) INTEL_MANUFACT:
            info->flash_id = FLASH_MAN_INTEL;
            printf("info->flash_id = %xWn", info->flash_id);
            break;

```

만약 인텔 플래시가 아닌 다른 회사 제품을 사용한다면 같은 플래시 메모리를 사용하는 레퍼런스 보드의 소스를 참조해서 위와 같은 형식으로 제조사를 체크하는 코드를 추가해야 한다. board 디렉토리에는 많은 레퍼런스 보드들의 코드가 있다. 여기에서 찾아보는 것이 좋다.

```

        default:
            printf("flash_get_size() failed.. %s %dWn", __FILE__, __LINE__);
            info->flash_id = FLASH_UNKNOWN;
            info->sector_count = 0;
            info->size = 0;
            addr[0] = (FPW) 0x00FF00FF;    /* restore read mode */
            return (0);                    /* no or unknown flash */
    }

```

```

        mb ();
        value = addr[1];                    /* device ID */

```

```

switch (value) {

case (FPW) INTEL_ID_28F128J3A:
    info->flash_id += FLASH_28F128J3A;
    info->sector_count = 128;
    info->size = 0x02000000;
    break;                                /* => 16 MB */

    제대로 플래시가 인식되었는지 확인한다. 부트로더가 실행되고 여기서 작성한 메시지들이
    출력되면 플래시 메모리가 정상적으로 인식되었다는 것을 확인할 수 있다. flash_info 구조체에
    include/configs/skku.h 에서 정의해준 플래시 메모리의 설정 값들을 저장한다.
    /* FIX BY GIO */
case (FPW) INTEL_ID_28F128K3:
    printf("28F128K3 is detected..Wn");
    info->flash_id += FLASH_28F128K3;
    info->sector_count = CFG_MAX_FLASH_SECT;
    info->size = CFG_FLASH_SIZE;
    printf("id = %x, sector_count = %x, size = %xWn",
           info->flash_id, info->sector_count, info->size);
    break;
default:
    info->flash_id = FLASH_UNKNOWN;
    break;
}

if (info->sector_count > CFG_MAX_FLASH_SECT) {
    printf ("** ERROR: sector count %d > max (%d) **Wn",
           info->sector_count, CFG_MAX_FLASH_SECT);
    info->sector_count = CFG_MAX_FLASH_SECT;
}

}

플래시 메모리의 장치 정보 읽기 동작을 끝낸다.
addr[0] = (FPW) 0xFFFFFFFF; /* 0x00FF00FF;*/ /* restore read mode */

return (info->size);
}

/*-----
--
*/
이 함수는 커널을 다운로드하기 전에 플래시 메모리를 지우기 위한 함수이다. 플래시 메모리의
제조사마다 코드가 달라질 수 있으므로 반드시 자신이 사용하는 플래시 메모리의 제조사와
일치하는지 확인해야 한다.
int flash_erase (flash_info_t *info, int s_first, int s_last)
{
    int flag, prot, sect;

```



```

ulong type, start, last;
int rcode = 0;

if ((s_first < 0) || (s_first > s_last)) {
    if (info->flash_id == FLASH_UNKNOWN) {
        printf ("- missingWn");
    } else {
        printf ("- no sectors to eraseWn");
    }
    return 1;
}

```

이 함수에서 플래시 메모리에 전송할 명령들의 코드는 인텔에서 정의한 것을 따랐으므로 다른 회사의 제품은 사용할 수 없다.

```

type = (info->flash_id & FLASH_VENDMASK);
if ((type != FLASH_MAN_INTEL)) {
    printf ("Can't erase unknown flash type %08lx - abortedWn",
        info->flash_id);
    return 1;
}

prot = 0;
for (sect = s_first; sect <= s_last; ++sect) {
    if (info->protect[sect]) {
        prot++;
    }
}

if (prot) {
    printf ("- Warning: %d protected sectors will not be erased!Wn",
        prot);
} else {
    printf ("Wn");
}

start = get_timer (0);
last = start;

/* Disable interrupts which might cause a timeout here */
flag = disable_interrupts ();

/* Start erase on unprotected sectors */
for (sect = s_first; sect <= s_last; sect++) {
    if (info->protect[sect] == 0) { /* not protected */
        FPWV *addr = (FPWV *) (info->start[sect]);
        FPW status;
    }
}

```

```

printf ("Erasing sector %2d ... ", sect);

/* arm simple, non interrupt dependent timer */
reset_timer_masked ();

*addr = (FPW) 0x00500050;      /* clear status register */
*addr = (FPW) 0x00200020;      /* erase setup */
*addr = (FPW) 0x00D000D0;      /* erase confirm */

while (((status = *addr) & (FPW) 0x00800080) != (FPW)
0x00800080) {
    if (get_timer_masked () > CFG_FLASH_ERASE_TOUT) {
        printf ("Timeout\n");
        *addr = (FPW) 0x00B000B0;      /* suspend erase
*/
        *addr = (FPW) 0x00FF00FF;      /* reset to read
mode */

        rcode = 1;
        break;
    }
}

*addr = 0x00500050;      /* clear status register cmd. */
*addr = 0xFFFFFFFF; /* 0x00FF00FF; */ /* reseat to read mode */

printf (" done\n");
    }
}
return rcode;
}

/*-----
--
* Copy memory to flash, returns:
* 0 - OK
* 1 - write timeout
* 2 - Flash not erased
* 4 - Flash not identified
*/

int write_buff (flash_info_t *info, uchar *src, ulong addr, ulong cnt)
{
    ulong cp, wp;
    FPW data;

```

```

    int count, i, l, rc, port_width;

    if (info->flash_id == FLASH_UNKNOWN) {
        return 4;
    }
    /* get lower word aligned address */
#ifdef FLASH_PORT_WIDTH16
    wp = (addr & ~1);
    port_width = 2;
#else
    wp = (addr & ~3);
    port_width = 4;
#endif

    /*
     * handle unaligned start bytes
     */
    if ((l = addr - wp) != 0) {
        data = 0;
        for (i = 0, cp = wp; i < l; ++i, ++cp) {
            data = (data << 8) | (*(uchar *) cp);
        }
        for (; i < port_width && cnt > 0; ++i) {
            data = (data << 8) | *src++;
            --cnt;
            ++cp;
        }
        for (; cnt == 0 && i < port_width; ++i, ++cp) {
            data = (data << 8) | (*(uchar *) cp);
        }

        if ((rc = write_data (info, wp, SWAP (data))) != 0) {
            return (rc);
        }
        wp += port_width;
    }

    /*
     * handle word aligned part
     */
    count = 0;
    while (cnt >= port_width) {
        data = 0;
        for (i = 0; i < port_width; ++i) {
            data = (data << 8) | *src++;
        }

```

```

        if ((rc = write_data (info, wp, SWAP (data))) != 0) {
            return (rc);
        }
        wp += port_width;
        cnt -= port_width;
        if (count++ > 0x800) {
            spin_wheel ();
            count = 0;
        }
    }

    if (cnt == 0) {
        return (0);
    }

    /*
     * handle unaligned tail bytes
     */
    data = 0;
    for (i = 0, cp = wp; i < port_width && cnt > 0; ++i, ++cp) {
        data = (data << 8) | *src++;
        --cnt;
    }
    for (; i < port_width; ++i, ++cp) {
        data = (data << 8) | *(uchar *) cp;
    }

    return (write_data (info, wp, SWAP (data)));
}

/*-----
--
* Write a word or halfword to Flash, returns:
* 0 - OK
* 1 - write timeout
* 2 - Flash not erased
*/
static int write_data (flash_info_t *info, ulong dest, FPW data)
{
    FPWV *addr = (FPWV *) dest;
    ulong status;
    int flag;

    /* Check if Flash is (sufficiently) erased */
    if ((*addr & data) != data) {
        printf ("not erased at %08lx (%lx)Wn", (ulong) addr, *addr);
    }

```

```

        return (2);
    }
    /* Disable interrupts which might cause a timeout here */
    flag = disable_interrupts ();

    *addr = (FPW) 0x00400040;      /* write setup */
    *addr = data;

    /* arm simple, non interrupt dependent timer */
    reset_timer_masked ();

    /* wait while polling the status register */
    while (((status = *addr) & (FPW) 0x00800080) != (FPW) 0x00800080) {
        if (get_timer_masked () > CFG_FLASH_WRITE_TOUT) {
            *addr = (FPW) 0x00FF00FF;      /* restore read mode */
            return (1);
        }
    }

    *addr = (FPW) 0xFFFFFFFF; /* 0x00FF00FF; */ /* restore read mode */

    return (0);
}

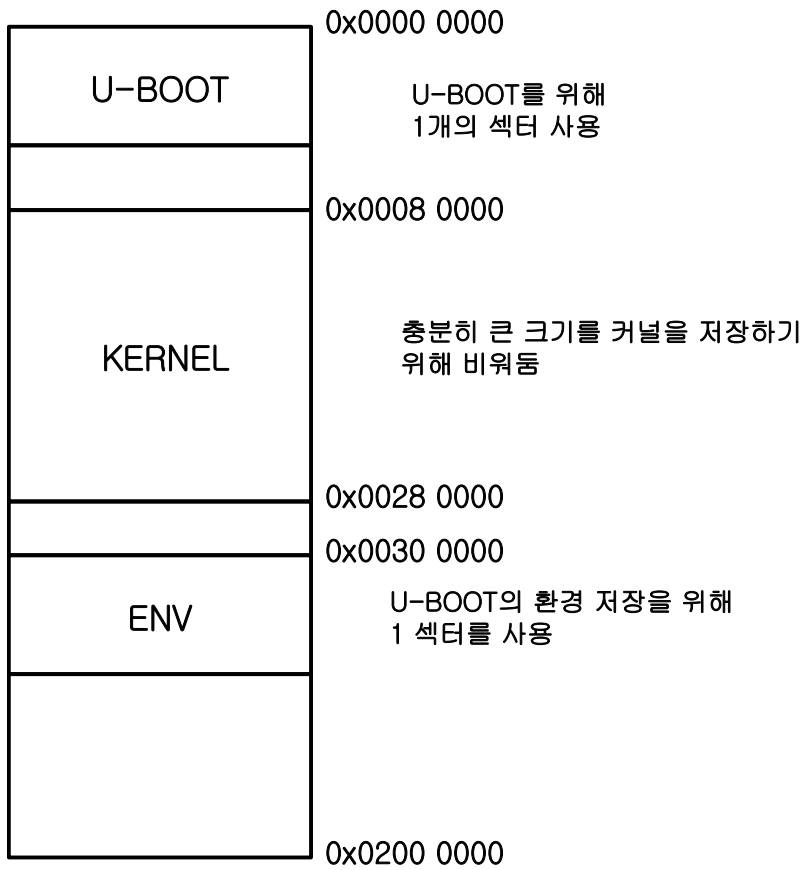
void inline spin_wheel (void)
{
    static int p = 0;
    static char w[] = "WW/-";

    printf ("W010%c", w[p]);
    (++p == 3) ? (p = 0) : 0;
}

```

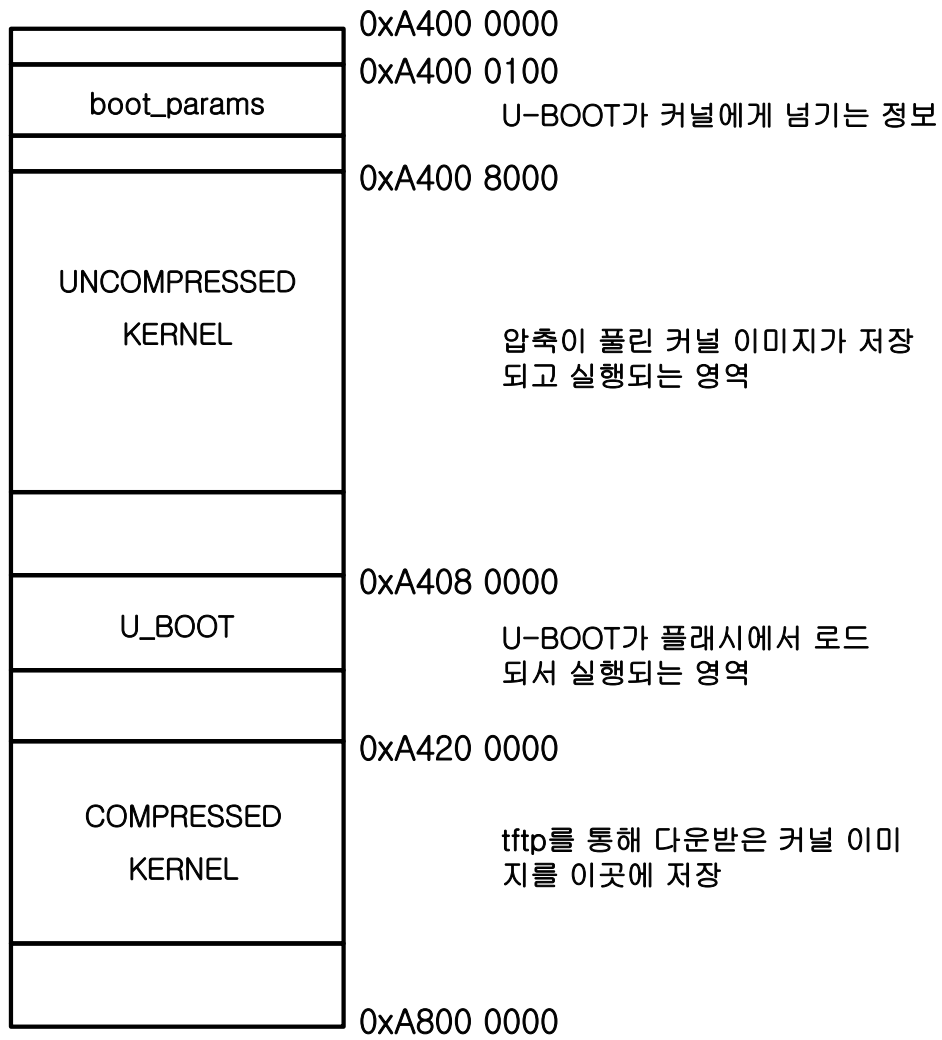
플래시 메모리 구조

플래시 메모리의 메모리 맵



SDRAM 메모리 구조

SDRAM의 메모리 맵



U-BOOT 사용하기

1. U-BOOT용 커널 이미지를 컴파일하기 위해서는 `make uImage` 명령으로 컴파일한다.
2. `tftp`로 커널 이미지를 전송하기 위해 `uImage` 파일을 `/tftpboot`로 복사한다.
3. 보드를 켜고 부트로더의 프롬프트가 나타나면 `tftp a4200000 uImage` 명령으로 커널 이미지를 전송받는다.
4. `bootm a4200000` 명령을 치면 U-BOOT는 커널 이미지가 압축되었는지 아닌지를 확인해서 압축을 풀고 압축이 풀어진 커널 이미지를 `a4008000`으로 복사한다. 그리고 `0xA4008000`으로 점프해서 커널을 시작한다.
5. 만약 `include/configs/sku.h` 에 `#define CONFIG_BOOTCOMMAND "tftp a4200000 uImage;bootm a4200000"` 라고 선언하면 이 과정을 자동으로 실행해준다.